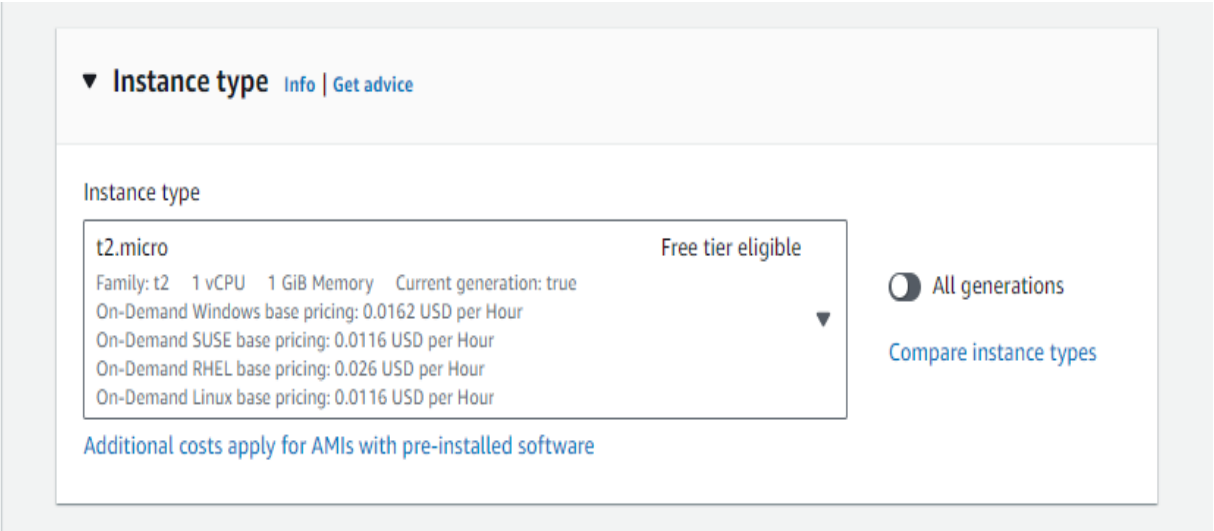
1. List of all EC2 instance parameters and their definition and description

Amazon EC2 (Elastic Compute Cloud) instances have a wide range of parameters that users can configure to meet specific needs. Here is a comprehensive list of some of the most important parameters, along with their definitions and descriptions:

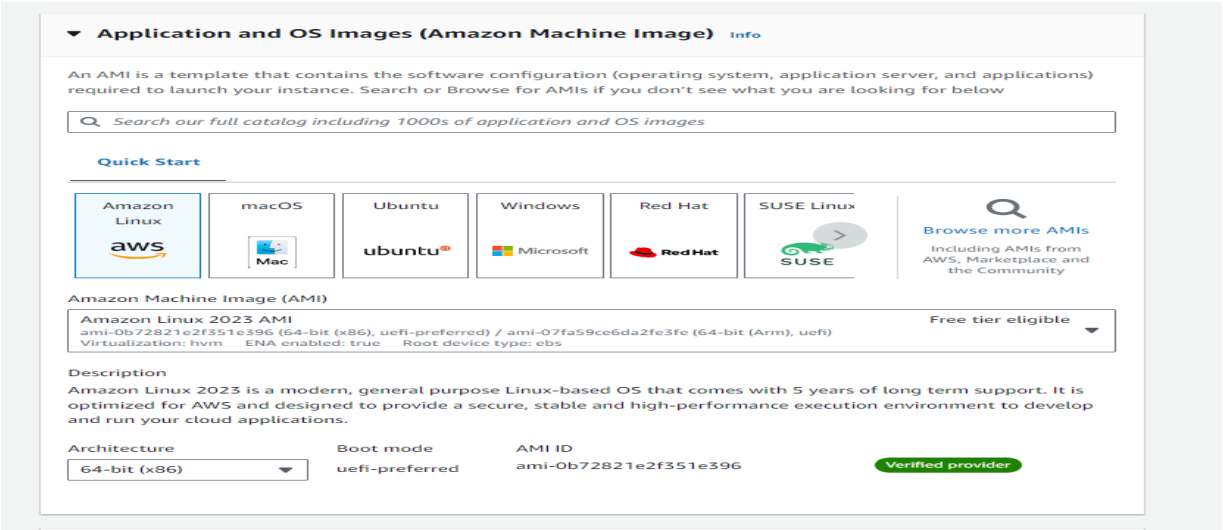
1. **Instance Type**: Specifies the hardware of the host computer used for the instance. Each instance type offers different compute, memory, and storage capabilities.

- Example: `t2.micro`, `m5.large`, `c5.xlarge`



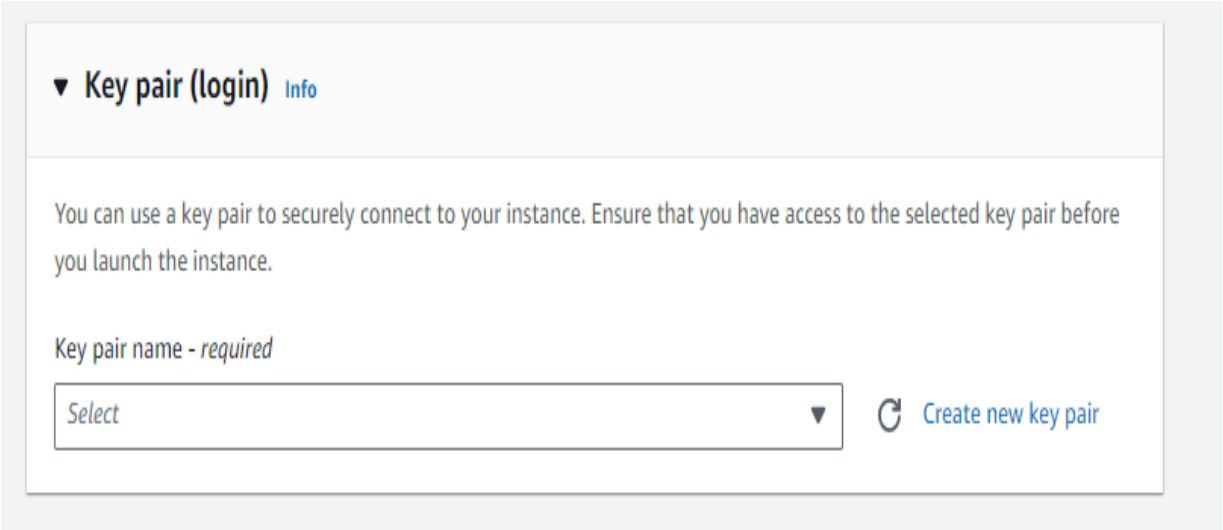
2. **AMI (Amazon Machine Image)**: A template that contains the software configuration (operating system, application server, applications) required to launch your instance.

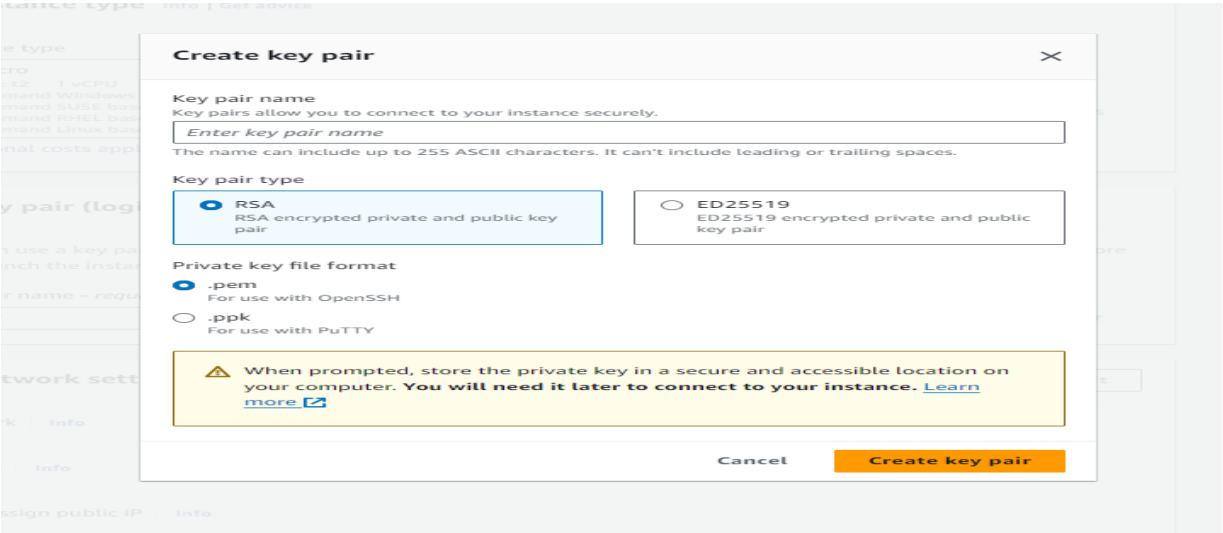
- Example: `ami-0abcdef1234567890`



3. **Key Pair**: A set of security credentials that you use to prove your identity when connecting to an EC2 instance.

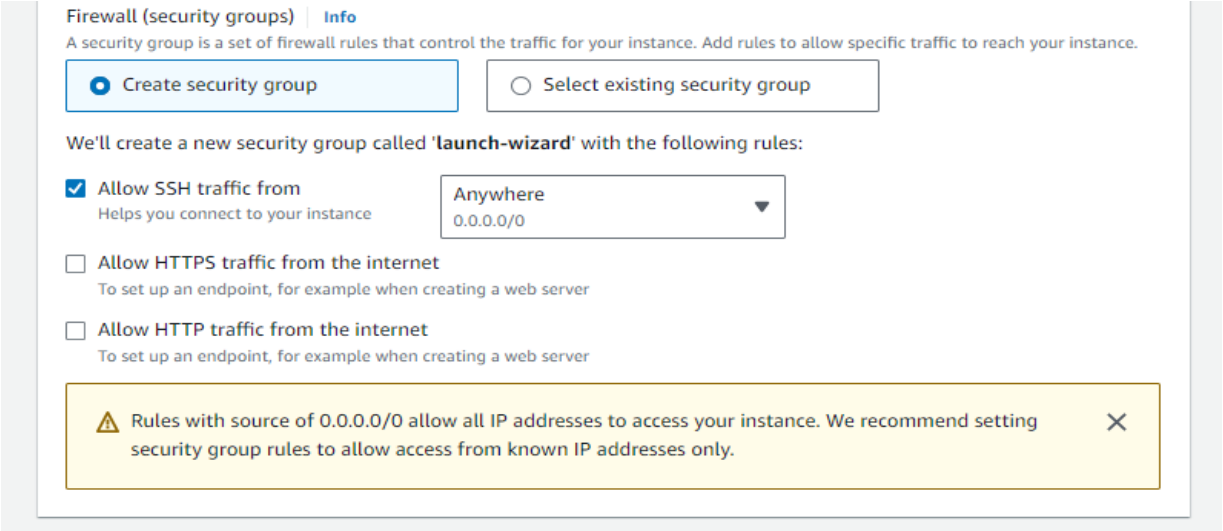
- Example: `my-key-pair`

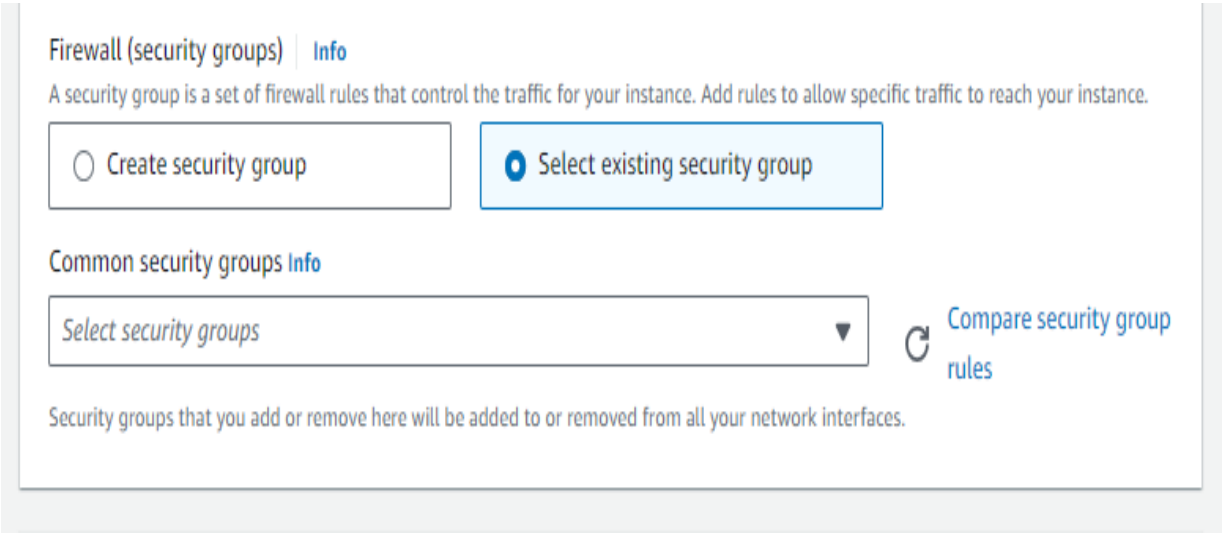




4. **Security Group**: A set of firewall rules that control the traffic for your instance. Security groups act at the instance level, not the subnet level.

- Example: `sg-123abc45`





5. **Instance ID**: A unique identifier for the instance.

   - Example: `i-0abcdef1234567890`

6. **Instance State**: The current state of the instance.

   - Example: `pending`, `running`, `stopping`, `stopped`

7. **Region and Availability Zone**: The geographic location where the instance is running. Regions are separate geographic areas, and Availability Zones are isolated locations within each region.

   - Example: `us-west-2a`

8. **Elastic IP**: A static IPv4 address designed for dynamic cloud computing. You can associate an Elastic IP address with any instance or network interface for any VPC in your account.

- Example: `54.123.45.67`

9. **Subnet**: A range of IP addresses in your VPC (Virtual Private Cloud). Instances in your VPC can communicate with each other, subject to security group and network ACL rules.

- Example: `subnet-123abc45`

10. **VPC (Virtual Private Cloud)**: A virtual network dedicated to your AWS account. It is logically isolated from other virtual networks in the AWS Cloud.

- Example: `vpc-123abc45`

11. **IAM Role**: An AWS Identity and Access Management (IAM) entity that defines a set of permissions for making AWS service requests.

- Example: `arn:aws:iam::123456789012:role/S3Access`

12. **Block Device Mapping**: Specifies the block devices (EBS volumes and instance store volumes) to attach to the instance when it's launched.

- Example: `/dev/sda1` mapped to `vol-123abc45`

13. **Tenancy**: Specifies whether your instance runs on shared hardware or single-tenant hardware.

- Example: `default`, `dedicated`

14. **Monitoring**: Indicates whether detailed monitoring is enabled for the instance. With detailed monitoring, Amazon CloudWatch collects monitoring data for your instance at a one-minute frequency.

- Example: `enabled`, `disabled`

15. **User Data**: Data that can be passed to the instance at launch. This data is often used to perform common automated configuration tasks and even run scripts after the instance starts.

- Example: Base64-encoded script data

16. **Placement Group**: Determines how instances are placed on underlying hardware. Can be used to influence the placement of a group of interdependent instances to meet the needs of your workload.

- Example: `cluster`, `spread`, `partition`

17. **Termination Protection**: When enabled, prevents the instance from being accidentally terminated.

- Example: `true`, `false`

18. **Shutdown Behavior**: Determines whether an instance should stop or terminate when an OS-level shutdown is performed.

- Example: `stop`, `terminate`

19. **Elastic GPUs**: Graphics processing units that you can attach to your Amazon EC2 instances to accelerate the graphics performance of your applications.

- Example: `eg1.medium`

20. **EBS-Optimized**: Provides additional, dedicated capacity for Amazon EBS I/O.

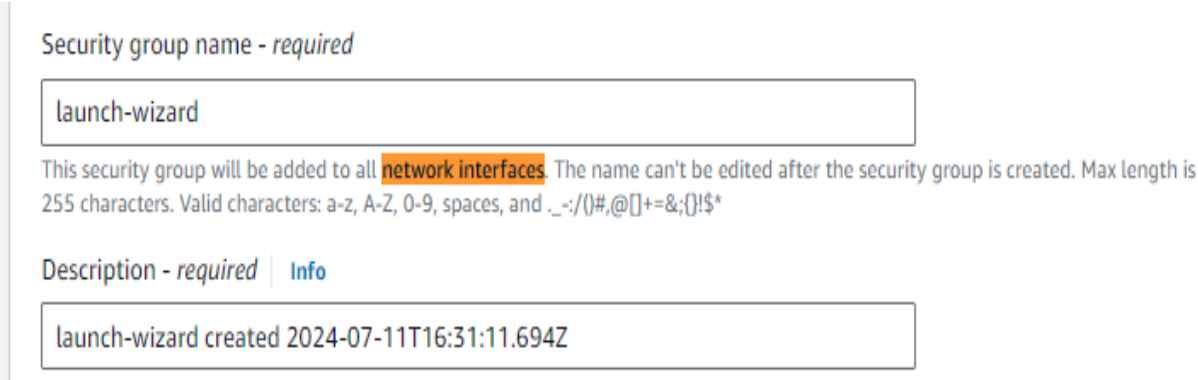
- Example: `true`, `false`

21. **Instance Lifecycle**: Indicates whether the instance is a spot instance or an on-demand instance.

- Example: `spot`, `on-demand`

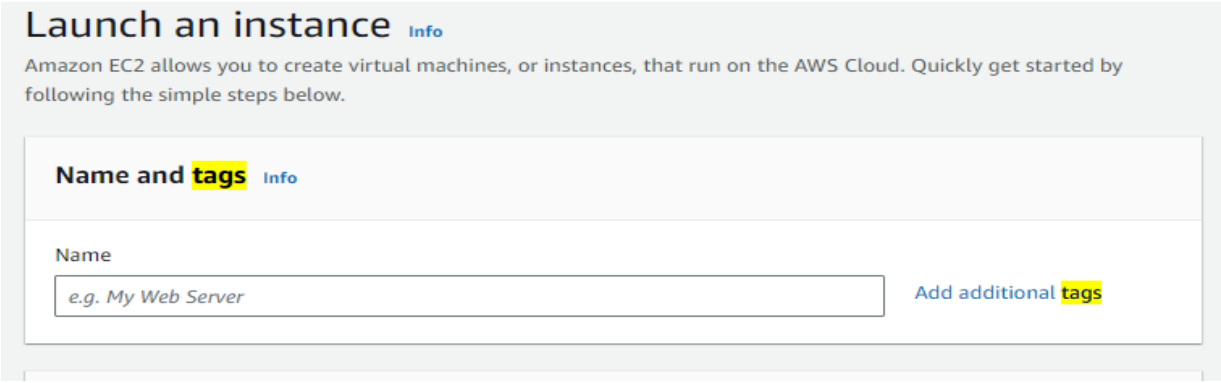
22. **Network Interfaces**: Specifies the network interfaces to attach to your instance.

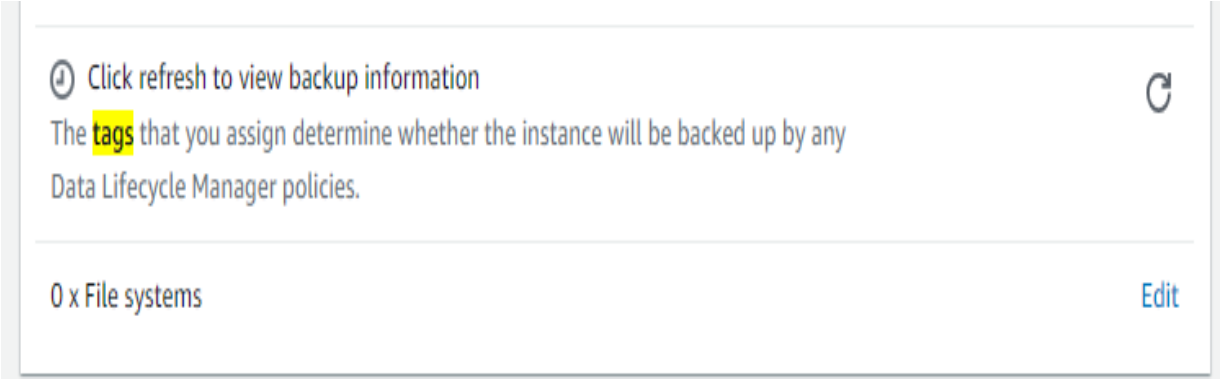
- Example: `eni-123abc45`



23. **Tags**: Key-value pairs that you can use to organize, search for, and identify your instances.

- Example: `Key=Name, Value=WebServer`





24. **Kernel ID**: The ID of the kernel associated with the instance.

- Example: `aki-123abc45`

25. **RAM Disk ID**: The ID of the RAM disk to be used with the instance.+99+

- Example: `ari-123abc45`

26. **Source/Destination Check**: Controls whether source/destination checking is enabled on the instance.

- Example: `true`, `false`

27. **Capacity Reservation**: Details about the instance's capacity reservation status.

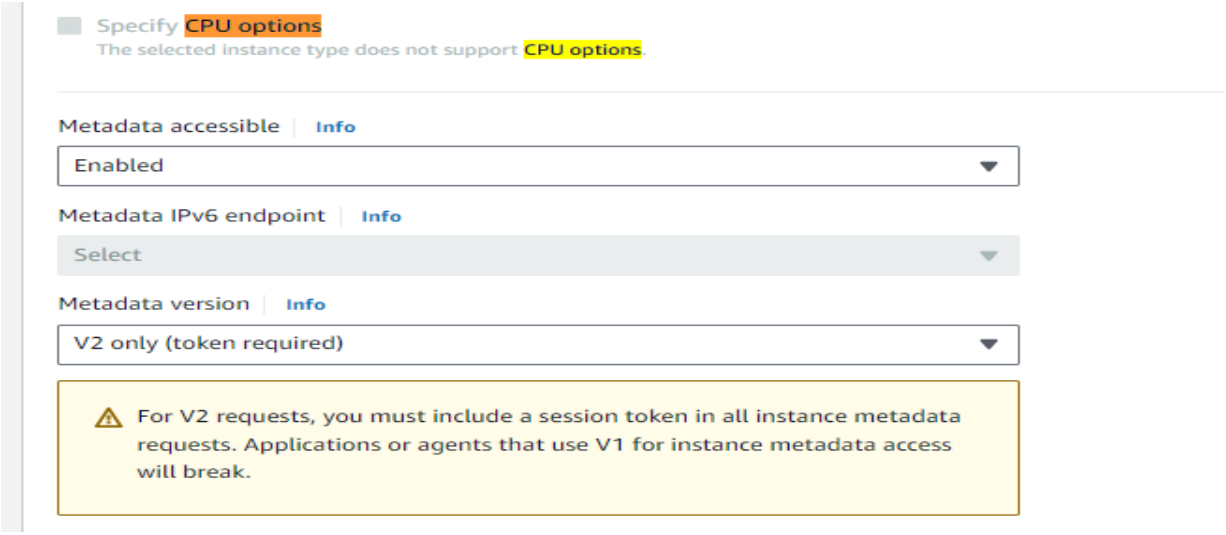
- Example: `open`, `targeted`

28. **Host ID**: The ID of the Dedicated Host if your instance is running on a Dedicated Host.

- Example: `host-123abc45`

29. **CPU Options**: The number of CPU cores and threads per core for the instance.

- Example: `CoreCount=4, ThreadsPerCore=2`



30. **Credit Specification**: The credit option for CPU usage (standard or unlimited) for burstable performance instances.

- Example: `standard`, `unlimited`

Each parameter can be configured through the AWS Management Console, AWS CLI, or AWS SDKs, providing flexibility in how you manage your EC2 instances.

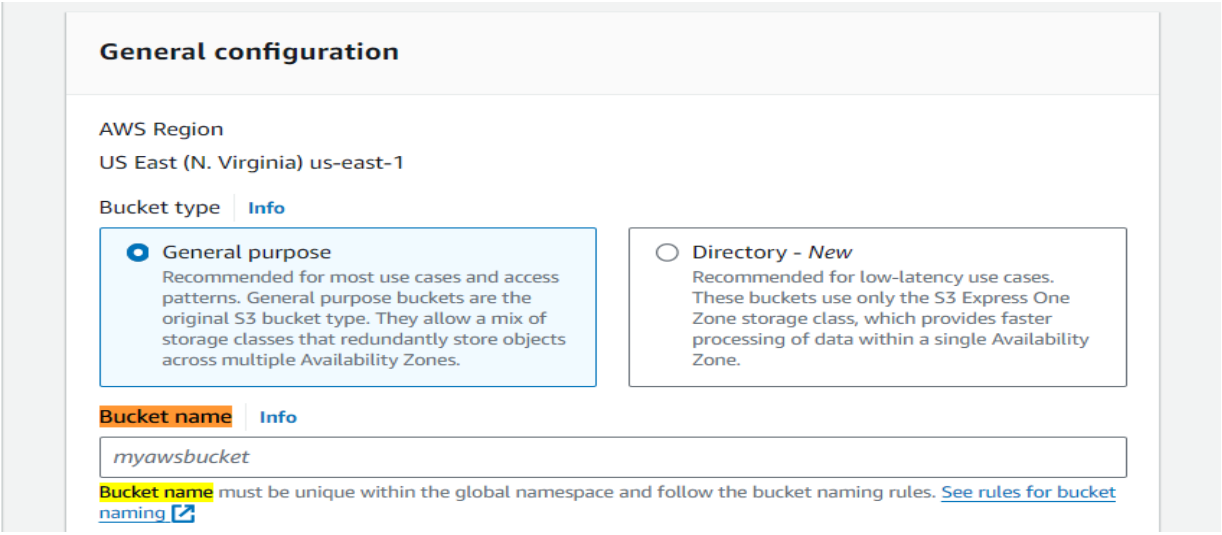
2. List of all S3 parameters and their definition and description

Amazon S3 (Simple Storage Service) provides a highly scalable, reliable, and low-latency data storage infrastructure. Below is a comprehensive list of S3 parameters, along with their definitions and descriptions:

Bucket-Level Parameters

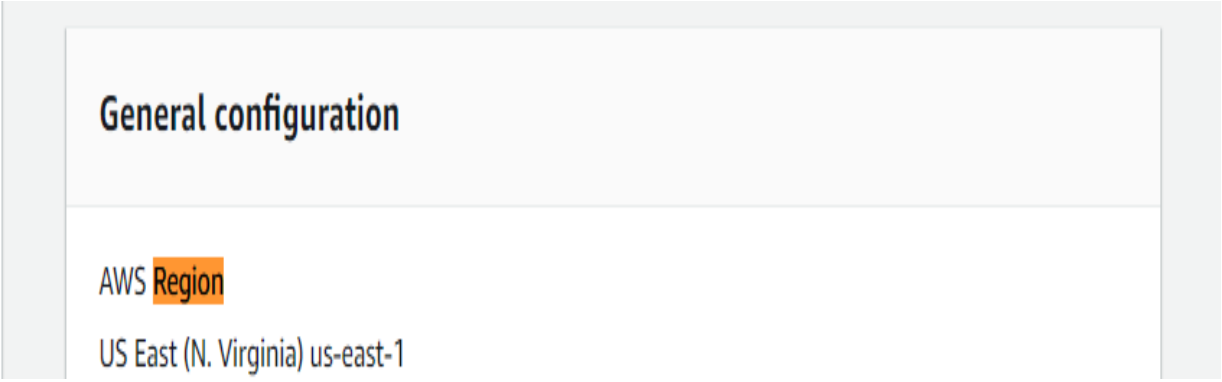
1. **Bucket Name**: A unique name assigned to the bucket. Bucket names must be unique across all of Amazon S3.

   - Example: `my-bucket`



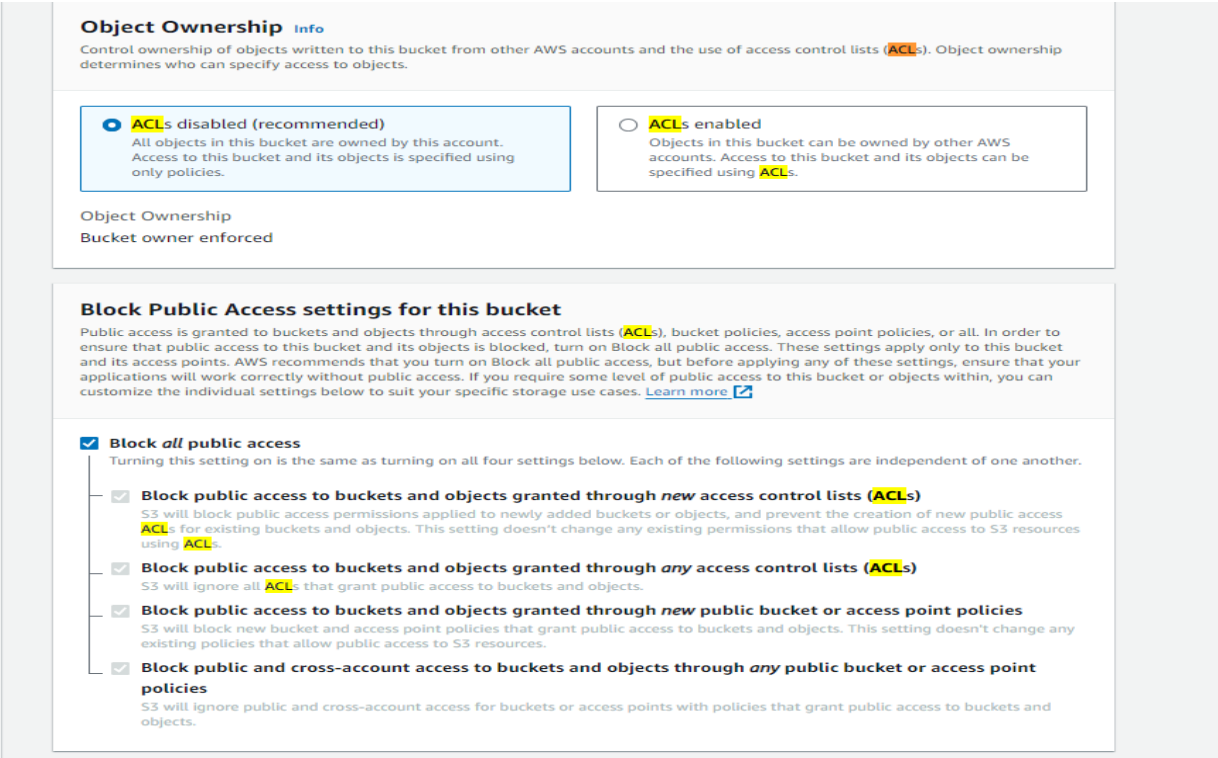
2. **Region**: The AWS region where the bucket is created.

   - Example: `us-west-2`



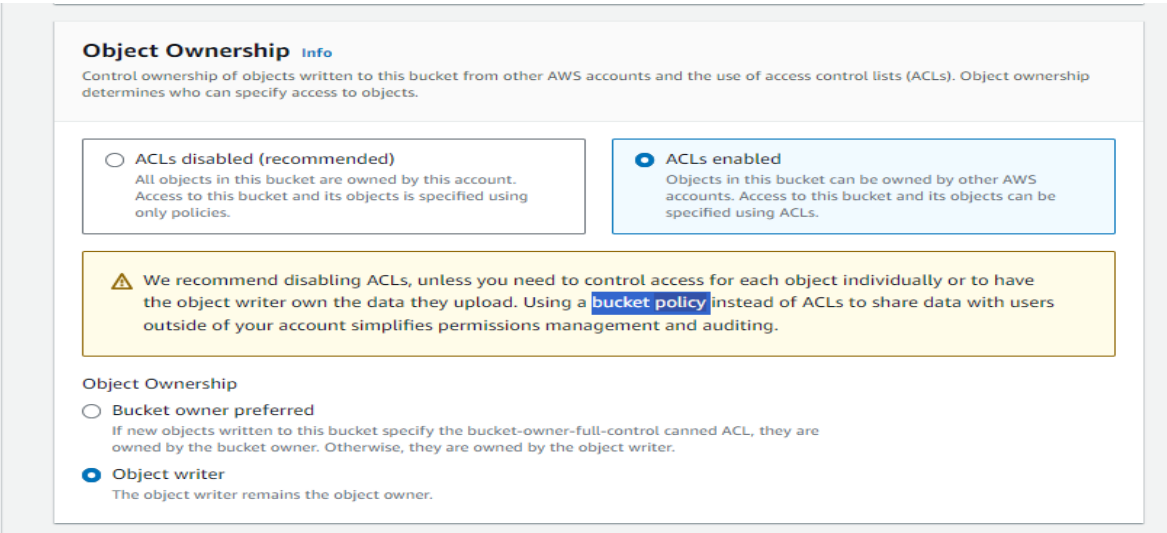
3. **ACL (Access Control List)**: Defines the access permissions for the bucket and the objects within it. ACLs can be set to grant access to specific AWS accounts or predefined groups.

- Example: `private`, `public-read`



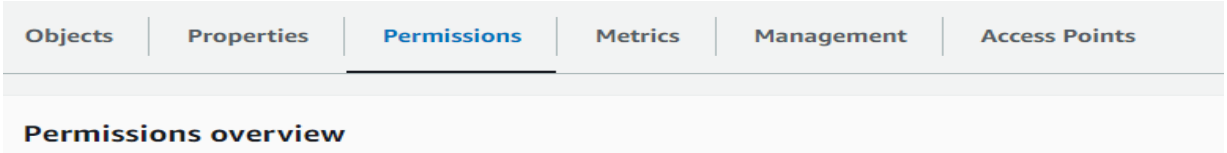
4. **Bucket Policy**: A JSON-based policy that defines access permissions for the bucket and the objects within it. Bucket policies provide finer control over permissions than ACLs.

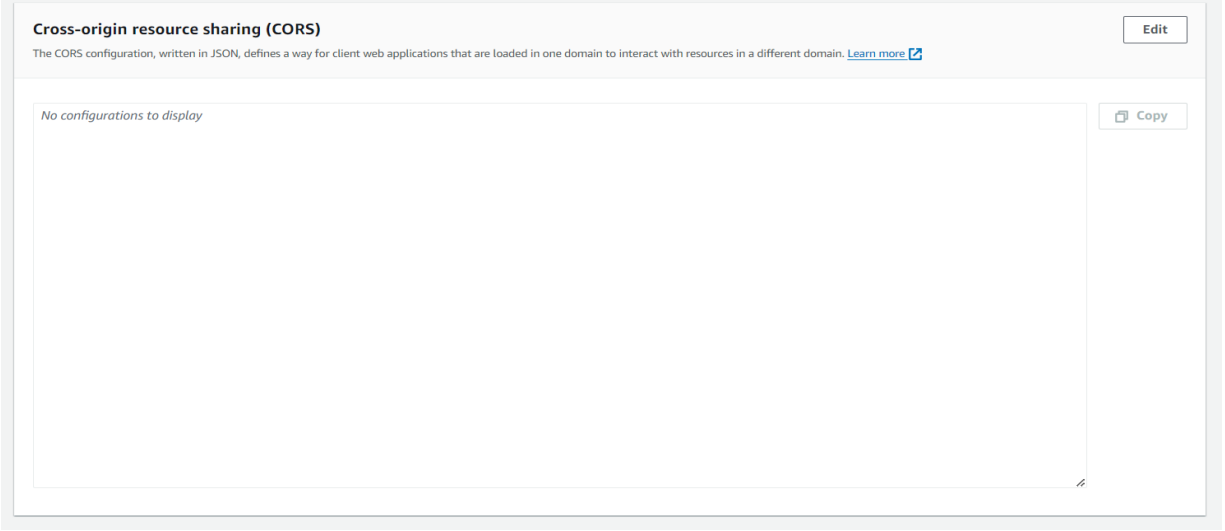
- Example: `{ "Version": "2012-10-17", "Statement": [ ... ] }`



5. **CORS (Cross-Origin Resource Sharing) Configuration**: Specifies the origins that are allowed to access the bucket, the HTTP methods that are supported, and other related configurations.

- Example: `{ "CORSRules": [ { "AllowedOrigins": [ "http://example.com" ], ... } ] }`

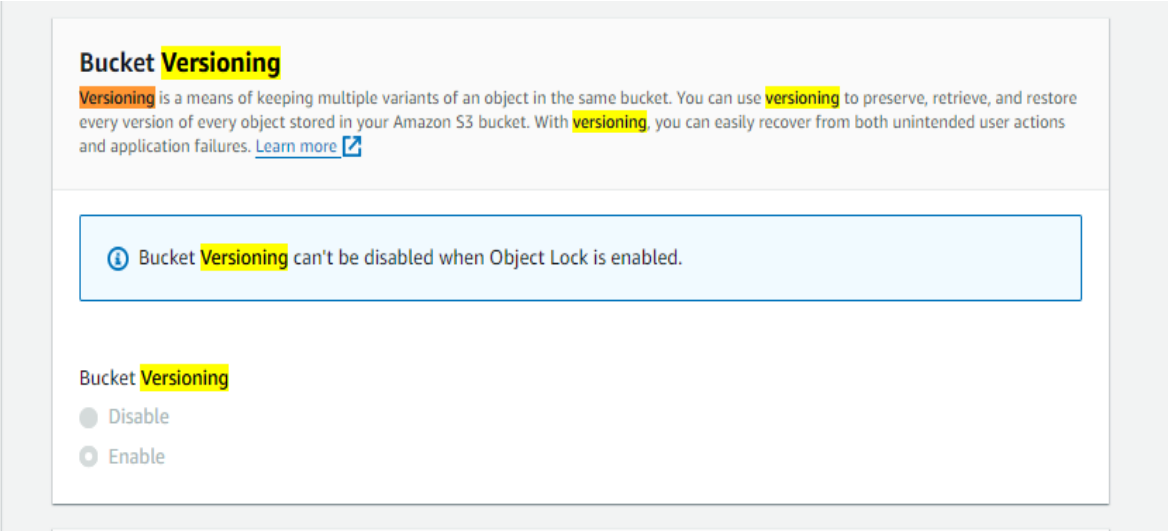




6. **Lifecycle Policy**: Defines rules for transitioning objects to different storage classes or deleting them after a specified period.

- Example: `{ "Rules": [ { "ID": "Rule1", "Status": "Enabled", ... } ] }`

7. **Versioning**: Enables or suspends versioning for the objects in the bucket, allowing multiple versions of objects to be stored.



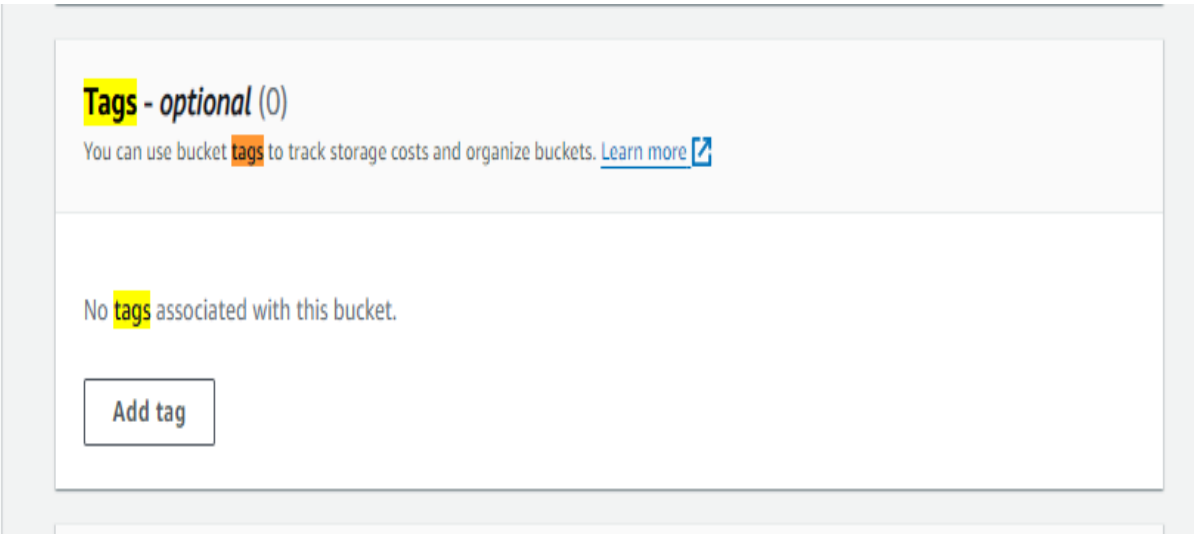
- Example: `Enabled`, `Suspended`

8. **Logging**: Enables server access logging, which provides detailed records for the requests made to the bucket.

- Example: `TargetBucket=my-logs-bucket, TargetPrefix=logs/`

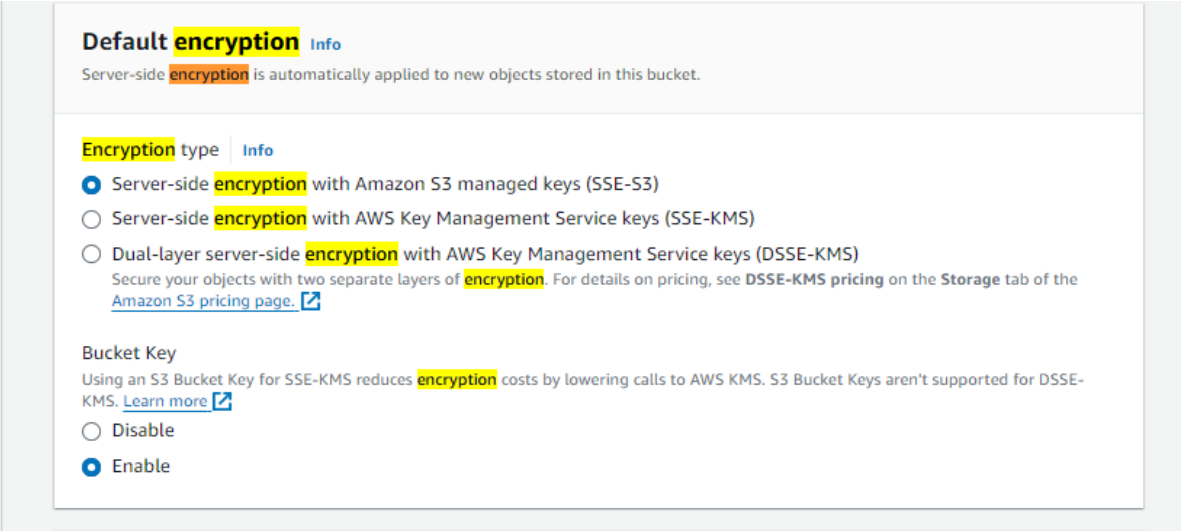
9. **Tags**: Key-value pairs that you can use to categorize your bucket.

- Example: `Key=Environment, Value=Production`



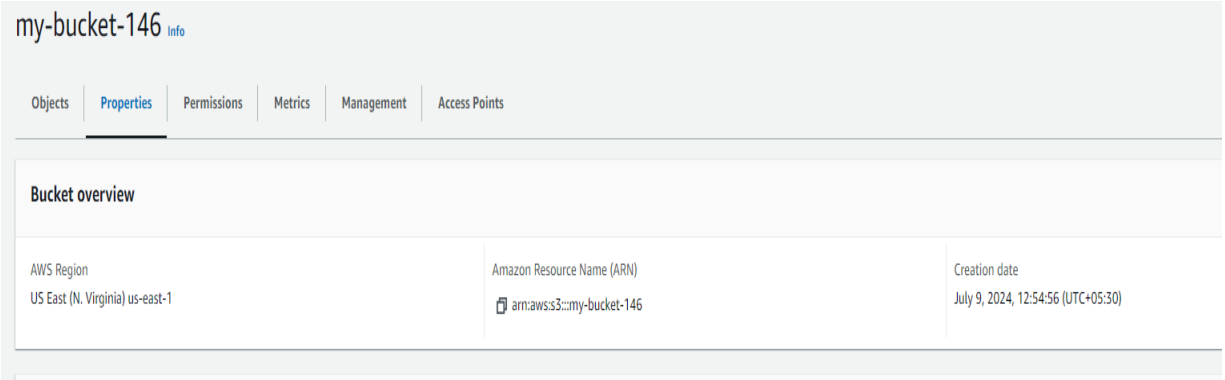
10. **Encryption**: Specifies whether objects in the bucket are automatically encrypted when stored.

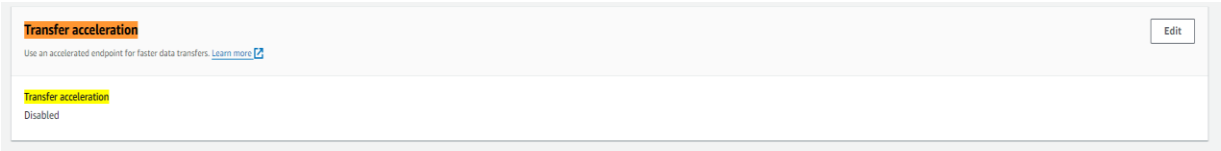
- Example: `SSE-S3`, `SSE-KMS`



11. **Transfer Acceleration**: Uses Amazon CloudFront's globally distributed edge locations to speed up uploads to S3.

- Example: `Enabled`, `Suspended`



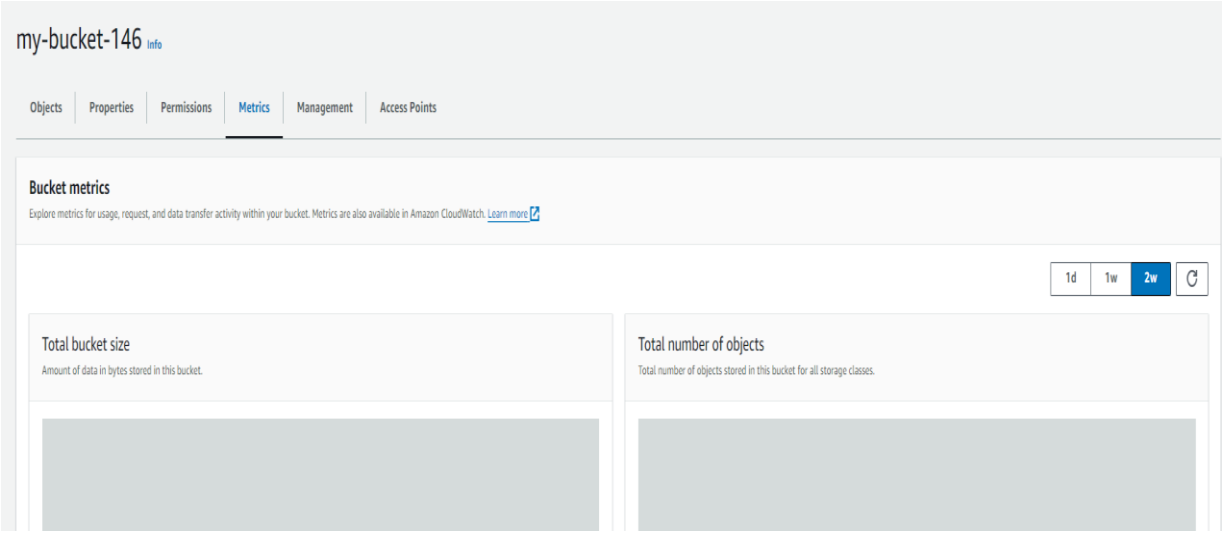


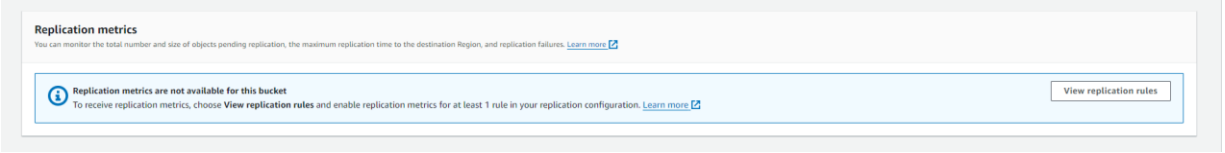
12. **Bucket Policy Conditions**: Specific conditions that must be met for requests to be authorized.

- Example: `{ "Condition": { "StringEquals": { "aws:Referer": "http://example.com" } } }`

13. **Replication**: Defines rules for replicating objects from one bucket to another, either within the same region or across different regions.

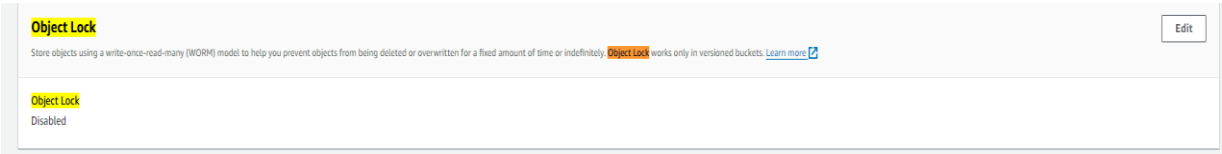
- Example: `{ "Rules": [ { "ID": "ReplicationRule1", ... } ] }`





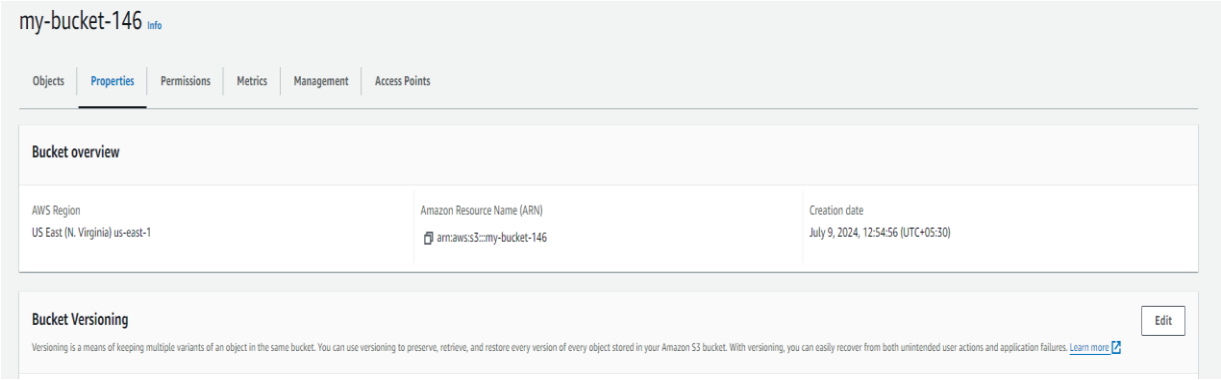
14. **Object Lock**: Protects objects from being deleted or overwritten for a fixed amount of time or indefinitely.

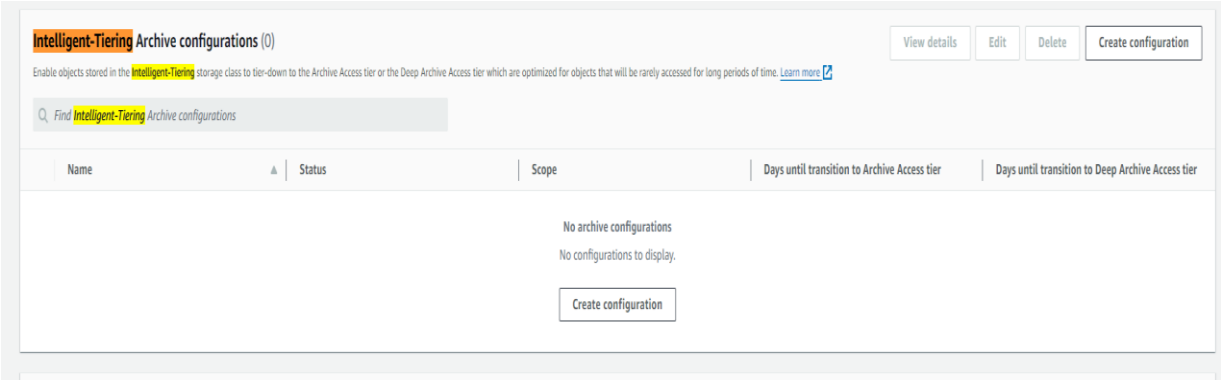
- Example: `Enabled`



15. **Intelligent-Tiering**: Automatically moves objects between two access tiers when access patterns change.

- Example: `Enabled`





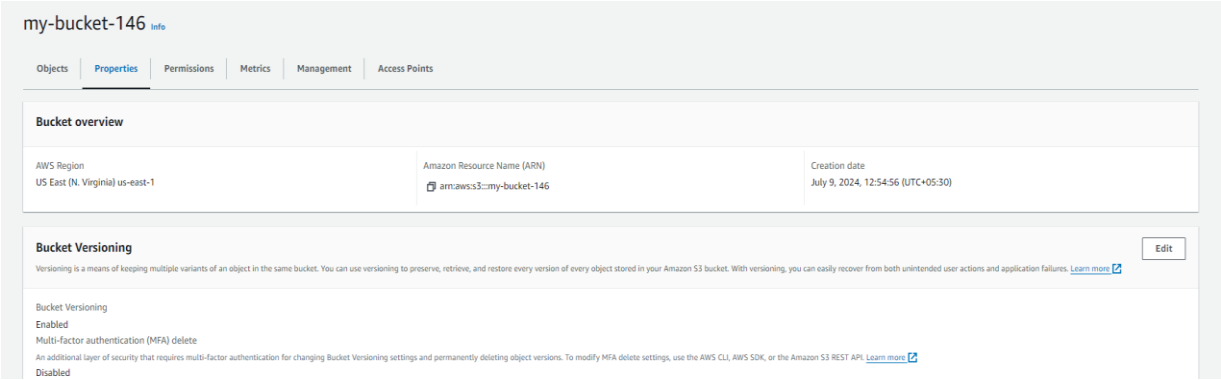
Object-Level Parameters

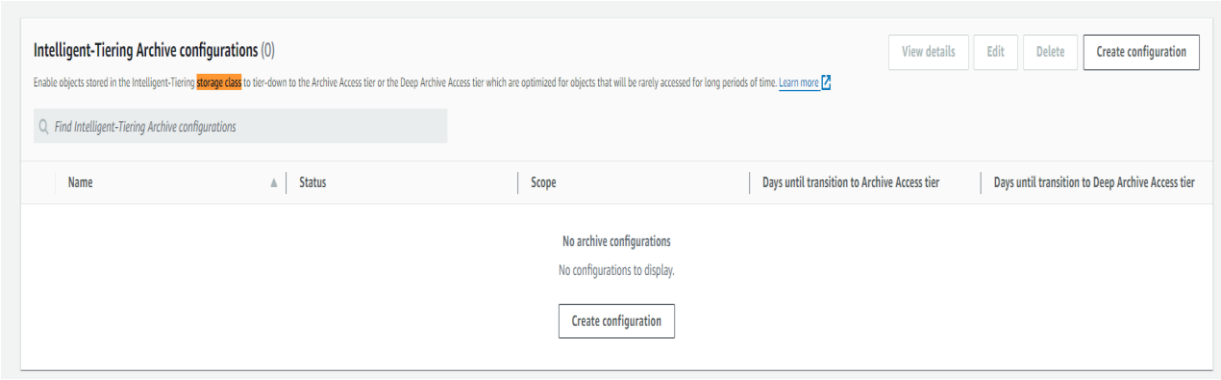
1. **Object Key**: The unique identifier for an object within a bucket.

   - Example: `photos/2021/01/01/my-photo.jpg`

2. **Storage Class**: The storage class of the object, determining the cost and performance characteristics.

   - Example: `STANDARD`, `INTELLIGENT_TIERING`, `GLACIER`





3. **Metadata**: Key-value pairs associated with the object. Metadata can be either system-defined (such as `Content-Type`) or user-defined.

   - Example: `{ "x-amz-meta-my-key": "my-value" }`

4. **Content-Type**: The MIME type of the object data.

   - Example: `image/jpeg`, `text/html`

5. **Content-Encoding**: The content encoding for the object data.

   - Example: `gzip`

6. **Content-Disposition**: Specifies presentational information for the object.

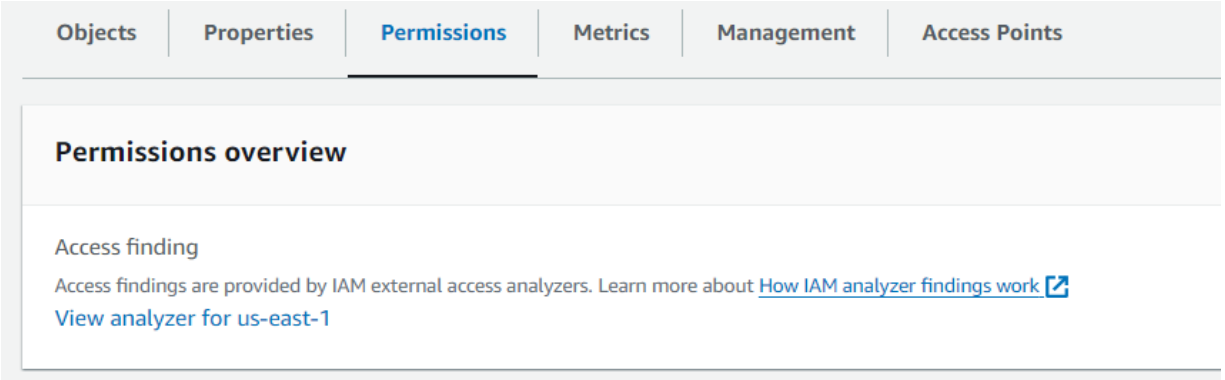
   - Example: `attachment; filename="filename.jpg"`

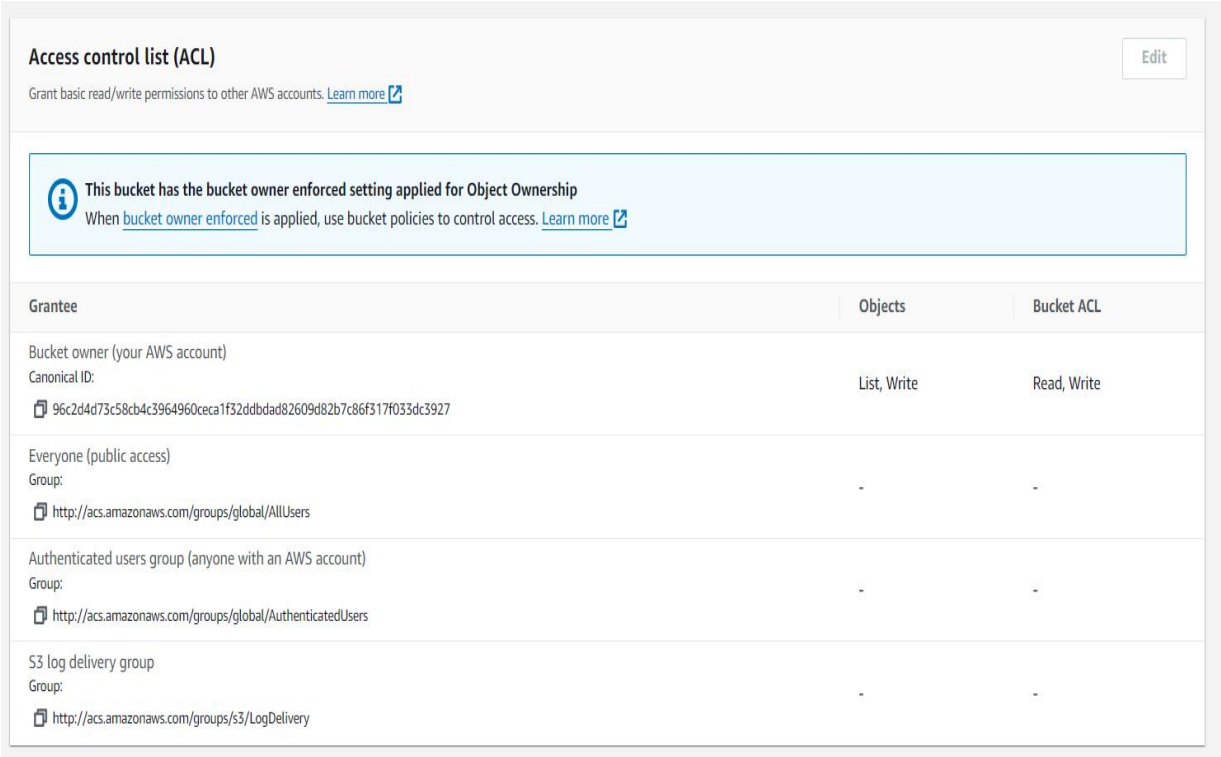
7. **Cache-Control**: Specifies caching behavior along the request/reply chain.

   - Example: `max-age=3600`

8. **ACL (Access Control List)**: Defines the access permissions for the object.

   - Example: `private`, `public-read`



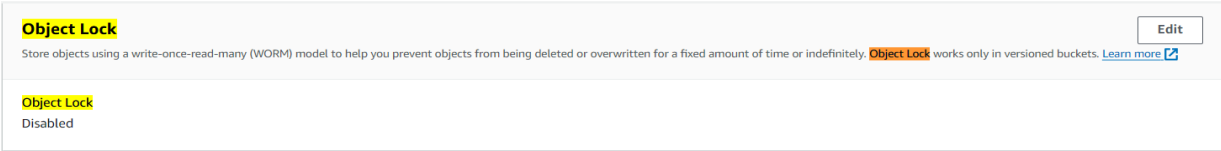


9. **Server-Side Encryption**: Indicates whether the object is encrypted on the server side.

   - Example: `AES256`, `aws:kms`

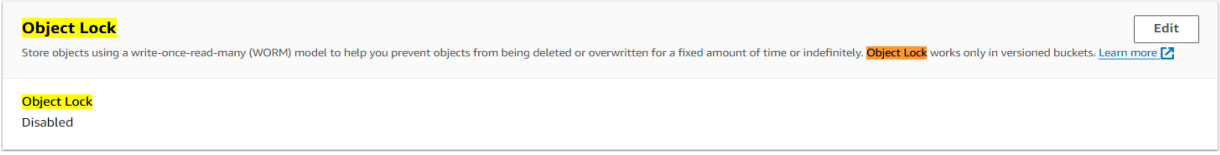
10. **Object Lock Retain Until Date**: The date until which the object is protected from being deleted or overwritten.

   - Example: `2022-12-31T23:59:59Z`



11. **Object Lock Legal Hold**: Indicates whether the object has a legal hold in place.

   - Example: `ON`



12. **Tagging**: Key-value pairs associated with the object for organization and management.

- Example: `Key=Project, Value=Project1`

13. **Checksum**: Provides the MD5, SHA-1, or other checksums for the object data to ensure data integrity.

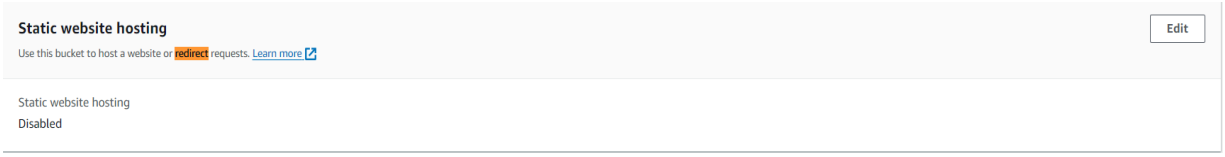
- Example: `e1a1f6db5f8f6c3db46f9f344efacd2d`

14. **Expires**: The date and time at which the object is no longer cacheable.

- Example: `Tue, 01 Jan 2030 00:00:00 GMT`

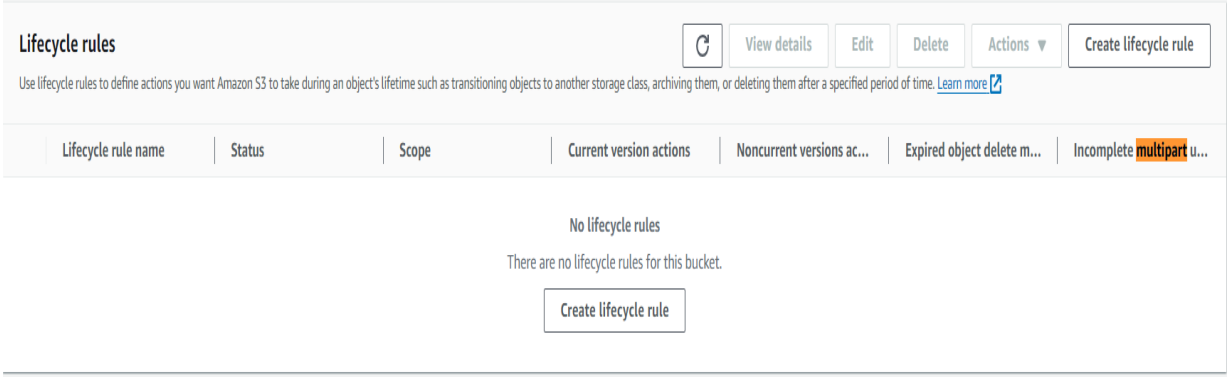
15. **Website Redirect Location**: The URL to which requests for the object are redirected.

- Example: `http://example.com`



16. **Multipart Upload**: Parameters associated with multipart upload processes, allowing large objects to be uploaded in parts.

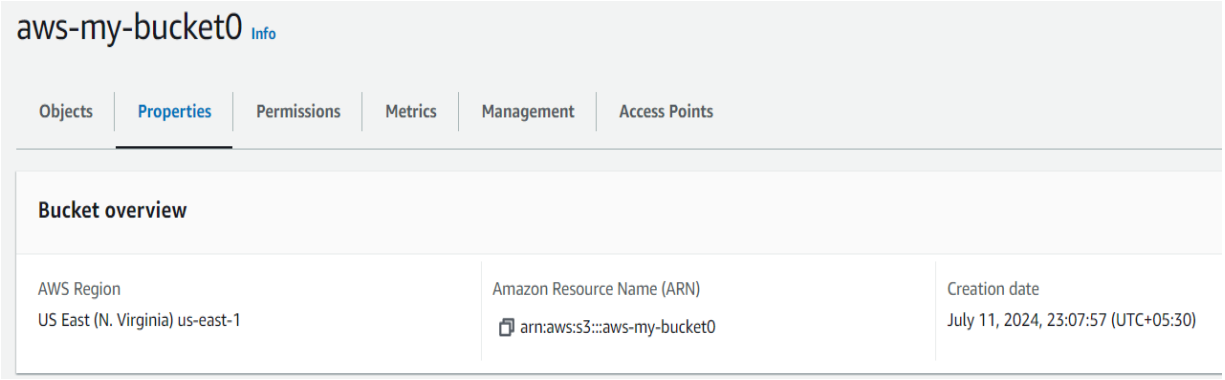
- Example: `UploadId=exampleUploadId, PartNumber=1`

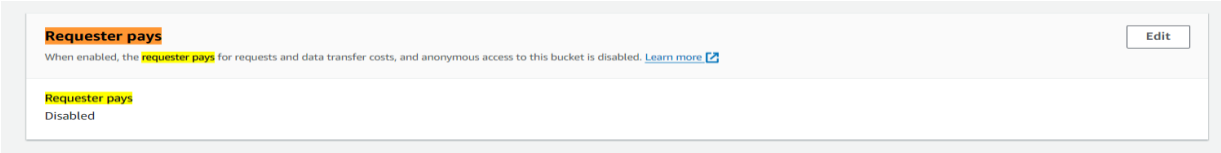


Access and Performance Parameters

1. **Requester Pays**: Specifies that the requester, rather than the bucket owner, pays for the data transfer.

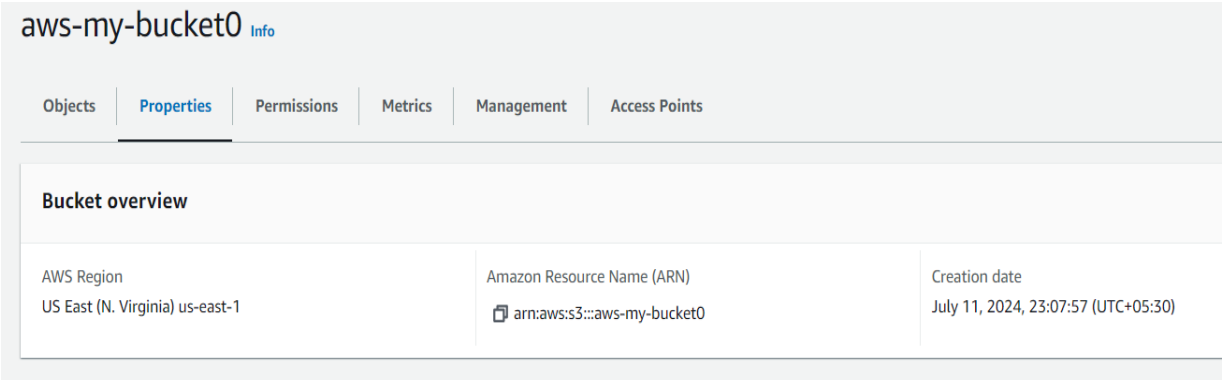
- Example: `Enabled`

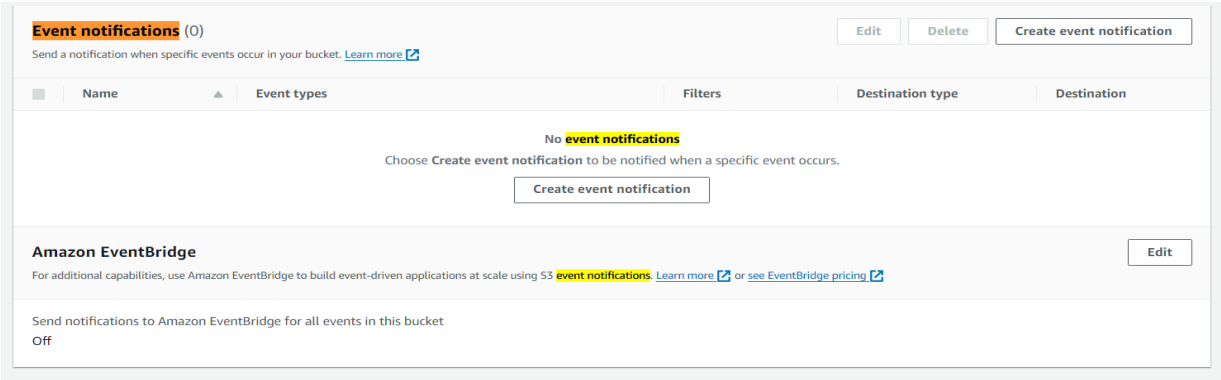




2. **Event Notifications**: Configures notifications to be sent in response to specific events (such as object creation or deletion).

- Example: `{ "LambdaFunctionConfigurations": [ { "LambdaFunctionArn": "arn:aws:lambda:us-west-2:123456789012:function:MyFunction", ... } ] }`





3. **Data Retrieval Policy**: Specifies the retrieval policy for archived objects in S3 Glacier.

- Example: `FreeTier`, `Standard`, `Bulk`

4. **Pre-Signed URLs**: Temporarily grants time-limited access to objects in your bucket.

- Example: URL generated with an expiration time.

5. **Access Points**: Provides a network endpoint to manage access to S3 buckets.

- Example: `arn:aws:s3:us-west-2:123456789012:accesspoint/my-access-point`

Each of these parameters can be configured through the AWS Management Console, AWS CLI, or AWS SDKs, offering flexibility in managing your S3 resources.

---

3. List of all RDS parameters and their definition and description

Amazon RDS (Relational Database Service) allows you to set up, operate, and scale a relational database in the cloud. Here is a comprehensive list of RDS instance parameters, along with their definitions and descriptions:

General Parameters

1. **DB Instance Identifier**: A unique name for your DB instance that is used to identify it.

- Example: `mydbinstance`

2. **DB Instance Class**: The compute and memory capacity of the DB instance. Instance classes are designed for different workloads.

- Example: `db.m5.large`

3. **Engine**: The database engine to be used for the DB instance.

- Example: `mysql`, `postgres`, `oracle-ee`

4. **Engine Version**: The version of the database engine.

- Example: `5.7.22`, `12.1.0.2.v10`

5. **DB Parameter Group**: A collection of engine configuration values that can be applied to one or more DB instances.

- Example: `default.mysql5.7`

6. **DB Cluster Parameter Group**: (For Aurora) A collection of engine configuration values that apply to a DB cluster.

   - Example: `default.aurora5.6`

Network & Security

7. **VPC**: The Virtual Private Cloud (VPC) in which your DB instance is located.

   - Example: `vpc-123abc45`

8. **Subnet Group**: A group of subnets that you can designate for your DB instances.

   - Example: `mydbsubnetgroup`

9. **Availability Zone**: The availability zone where your DB instance is deployed.

   - Example: `us-west-2a`

10. **Multi-AZ Deployment**: Specifies if the DB instance is a Multi-AZ deployment, which enhances availability and data durability.

    - Example: `true`, `false`

11. **Security Groups**: Specifies the security groups to be associated with the DB instance.

    - Example: `sg-123abc45`

12. **Publicly Accessible**: Specifies whether the DB instance is publicly accessible.

    - Example: `true`, `false`

13. **Endpoint**: The connection endpoint for the DB instance.

    - Example: `mydbinstance.c9akciq32.rds.amazonaws.com`

14. **Port**: The port on which the DB instance listens.

    - Example: `3306`

Storage

15. **Allocated Storage**: The amount of storage allocated to the DB instance.

   - Example: `100` (in GiB)

16. **Storage Type**: The storage type for the DB instance.

   - Example: `gp2` (General Purpose SSD), `io1` (Provisioned IOPS SSD)

17. **Iops**: The number of I/O operations per second (IOPS) for the DB instance (applicable for Provisioned IOPS SSD).

   - Example: `1000`

18. **Storage Encrypted**: Specifies whether the DB instance is encrypted.

   - Example: `true`, `false`

19. **KMS Key ID**: The AWS Key Management Service (KMS) key identifier for an encrypted DB instance.

   - Example: `arn:aws:kms:us-west-2:123456789012:key/abc123de-45fg-67hi-89jk-lmnopqrstuv`

Backup & Maintenance

20. **Backup Retention Period**: The number of days to retain automated backups.

   - Example: `7`

21. **Preferred Backup Window**: The daily time range during which automated backups are created.

   - Example: `07:00-09:00`

22. **Preferred Maintenance Window**: The weekly time range during which system maintenance can occur.

   - Example: `sun:05:00-sun:06:00`

23. **Auto Minor Version Upgrade**: Indicates that minor engine upgrades are applied automatically during the maintenance window.

- Example: `true`, `false`

Performance & Monitoring

24. **DB Instance Status**: The current status of the DB instance.
    - Example: `available`, `modifying`, `backing-up`

25. **DB Parameter Group**: The parameter group associated with the DB instance.
    - Example: `default.mysql5.7`

26. **Enhanced Monitoring**: Specifies whether enhanced monitoring is enabled for the DB instance.
    - Example: `enabled`, `disabled`

27. **Monitoring Interval**: The interval in seconds between points when Enhanced Monitoring metrics are collected.
    - Example: `60`

28. **Monitoring Role ARN**: The ARN of the IAM role that permits RDS to send enhanced monitoring metrics to CloudWatch Logs.
    - Example: `arn:aws:iam::123456789012:role/emaccess`

Replication

29. **Read Replica Source DB Instance Identifier**: The identifier of the source DB instance if this DB instance is a read replica.
    - Example: `mydbinstance-source`

30. **Read Replica DB Instance Identifiers**: The identifiers of any read replicas associated with this DB instance.
    - Example: `[ "mydbinstance-replica1", "mydbinstance-replica2" ]`

31. **Replication State**: The state of the replication.

- Example: `replicating`

32. **Replication Role**: The role of the DB instance in the replication configuration.
    - Example: `source`, `replica`

Parameters for Aurora DB Cluster

33. **DB Cluster Identifier**: A unique identifier for the DB cluster.
    - Example: `mydbcluster`

34. **DB Cluster Endpoint**: The endpoint for the DB cluster.
    - Example: `mydbcluster.cluster-c9akciq32.rds.amazonaws.com`

35. **DB Cluster Members**: The instances that are part of the DB cluster.
    - Example: `[ "mydbinstance1", "mydbinstance2" ]`

36. **Reader Endpoint**: The endpoint for the reader instances in the DB cluster.
    - Example: `mydbcluster.cluster-ro-c9akciq32.rds.amazonaws.com`

37. **Cluster Create Time**: The time when the DB cluster was created.
    - Example: `2021-01-01T12:00:00Z`

Database Specific Parameters

38. **Character Set Name**: The character set to associate with the DB instance.
    - Example: `utf8`

39. **Time Zone**: The time zone of the DB instance.
    - Example: `UTC`

40. **Option Group**: The option group associated with the DB instance.
    - Example: `default:mysql-5-7`

41. **Parameter Group**: The parameter group associated with the DB instance.

- Example: `default.mysql5.7`

42. **Copy Tags To Snapshot**: Specifies whether tags are copied from the DB instance to snapshots.

- Example: `true`, `false`

Each parameter can be configured through the AWS Management Console, AWS CLI, or AWS SDKs, providing flexibility in managing your RDS instances.

---

4. List of all DynamoDB parameters and their definition and description

Amazon DynamoDB is a fully managed NoSQL database service that provides fast and predictable performance with seamless scalability. Here is a comprehensive list of DynamoDB table parameters, along with their definitions and descriptions:

General Table Parameters

1. **Table Name**: The name of the DynamoDB table. Table names must be unique within an AWS account and region.

- Example: `Users`

2. **Primary Key**: Defines the unique identifier for each item in the table. This can be a single attribute (partition key) or a composite key (partition key and sort key).

- Example: `UserId` (Partition Key) or `UserId` and `OrderId` (Partition Key and Sort Key)

Capacity and Scaling

3. **Provisioned Throughput**: The read and write capacity units allocated to the table.

- Example: `{ ReadCapacityUnits: 5, WriteCapacityUnits: 5 }`

4. **Auto Scaling**: Automatically adjusts the provisioned throughput of the table based on traffic patterns.

- Example: `Enabled`

5. **Read Capacity Units (RCU)**: The number of strongly consistent reads per second, or eventually consistent reads per second (twice as many) that the table can handle.

   - Example: `5`

6. **Write Capacity Units (WCU)**: The number of writes per second that the table can handle.

   - Example: `5`

Indexes

7. **Global Secondary Index (GSI)**: An index with a partition key and optional sort key that can be different from those on the table.

   - Example: `{ IndexName: 'UserIndex', KeySchema: [ { AttributeName: 'UserId', KeyType: 'HASH' } ], ProvisionedThroughput: { ReadCapacityUnits: 5, WriteCapacityUnits: 5 } }`

8. **Local Secondary Index (LSI)**: An index that shares the partition key of the table but has a different sort key.

   - Example: `{ IndexName: 'OrderIndex', KeySchema: [ { AttributeName: 'UserId', KeyType: 'HASH' }, { AttributeName: 'OrderDate', KeyType: 'RANGE' } ] }`

Data Types

9. **Attribute Definitions**: Specifies the names and data types of the attributes in the primary key and indexes.

   - Example: `[ { AttributeName: 'UserId', AttributeType: 'S' }, { AttributeName: 'OrderId', AttributeType: 'N' } ]`

10. **Attribute Names**: The names of attributes in the table.

    - Example: `UserId`, `OrderId`

11. **Attribute Types**: The data types of the attributes.

    - Example: `S` (String), `N` (Number), `B` (Binary)

Performance and Storage

12. **Item Size**: The size of an item in the table, including attribute names and values.
   - Example: `400 KB`

13. **Storage Type**: Indicates whether the table uses standard or on-demand capacity mode.
   - Example: `Provisioned`, `On-Demand`

Security and Access Control

14. **IAM Policies**: IAM policies that specify who can access the table and what actions they can perform.
   - Example: `arn:aws:iam::123456789012:policy/DynamoDBAccessPolicy`

15. **KMS Key**: The AWS Key Management Service (KMS) key used to encrypt the table.
   - Example: `arn:aws:kms:us-west-2:123456789012:key/abc123de-45fg-67hi-89jk-lmnopqrstuv`

16. **Encryption at Rest**: Indicates whether the data is encrypted at rest using AWS KMS.
   - Example: `Enabled`

Backup and Restore

17. **Backup and Restore**: Options for backing up and restoring the table data.
   - Example: `Enabled`

18. **Point-in-Time Recovery (PITR)**: Enables continuous backups and the ability to restore the table to any point in time within the last 35 days.
   - Example: `Enabled`

Streams

19. **DynamoDB Streams**: Captures data modification events in the table and delivers them to a stream.

- Example: `Enabled`

20. **Stream View Type**: The information that the stream captures when an item in the table is modified.
    - Example: `NEW_IMAGE`, `OLD_IMAGE`, `NEW_AND_OLD_IMAGES`, `KEYS_ONLY`

Tags and Metadata

21. **Tags**: Key-value pairs that you can use to categorize and manage your DynamoDB tables.
    - Example: `{ Key: 'Environment', Value: 'Production' }`

22. **TTL (Time to Live)**: Defines a timestamp attribute to automatically delete expired items from the table.
    - Example: `{ AttributeName: 'ExpirationTime', Enabled: true }`

Global Tables

23. **Global Table**: A collection of one or more replica tables, in different regions, that DynamoDB treats as a single unit.
    - Example: `Enabled`

24. **Replication Regions**: The AWS regions where the replica tables are located.
    - Example: `[ 'us-west-2', 'us-east-1' ]`

Miscellaneous

25. **Billing Mode**: Indicates whether the table uses provisioned or on-demand capacity mode.
    - Example: `PROVISIONED`, `PAY_PER_REQUEST`

26. **Continuous Backups**: Indicates whether continuous backups are enabled for the table.
    - Example: `Enabled`

27. **Stream ARN**: The Amazon Resource Name (ARN) of the DynamoDB stream.

- Example: `arn:aws:dynamodb:us-west-2:123456789012:table/Users/stream/2021-01-01T00:00:00.000`

These parameters can be configured through the AWS Management Console, AWS CLI, or AWS SDKs, providing flexibility in managing your DynamoDB tables.

---

5. List of all VPC parameters and their definition and description

Amazon VPC (Virtual Private Cloud) allows you to provision a logically isolated section of the AWS cloud where you can launch AWS resources in a virtual network that you define. Here is a comprehensive list of VPC parameters, along with their definitions and descriptions:

General VPC Parameters

1. **VPC ID**: The unique identifier for the VPC.
   - Example: `vpc-123abc45`

2. **VPC Name**: The name assigned to the VPC.
   - Example: `my-vpc`

3. **CIDR Block**: The IP address range for the VPC, in CIDR notation.
   - Example: `10.0.0.0/16`

4. **IPv6 CIDR Block**: The IPv6 address range for the VPC, in CIDR notation.
   - Example: `2001:db8:1234:1a00::/56`

5. **Tenancy**: The tenancy option for instances launched into the VPC.
   - Example: `default`, `dedicated`

Subnets

6. **Subnet ID**: The unique identifier for a subnet within the VPC.
   - Example: `subnet-123abc45`

7. **Subnet Name**: The name assigned to the subnet.

   - Example: `my-subnet`

8. **Subnet CIDR Block**: The IP address range for the subnet, in CIDR notation.

   - Example: `10.0.1.0/24`

9. **Availability Zone**: The availability zone in which the subnet resides.

   - Example: `us-west-2a`

10. **Auto-assign Public IP**: Indicates whether instances launched in this subnet receive a public IP address.

    - Example: `true`, `false`

Route Tables

11. **Route Table ID**: The unique identifier for the route table.

    - Example: `rtb-123abc45`

12. **Routes**: The routing rules associated with the route table.

    - Example: `[ { "DestinationCidrBlock": "0.0.0.0/0", "GatewayId": "igw-123abc45" } ]`

13. **Associations**: The subnets associated with the route table.

    - Example: `[ "subnet-123abc45", "subnet-678def90" ]`

Internet Gateways

14. **Internet Gateway ID**: The unique identifier for the internet gateway.

    - Example: `igw-123abc45`

15. **Internet Gateway Name**: The name assigned to the internet gateway.

    - Example: `my-igw`

NAT Gateways

16. **NAT Gateway ID**: The unique identifier for the NAT gateway.
   - Example: `nat-123abc45`

17. **NAT Gateway Name**: The name assigned to the NAT gateway.
   - Example: `my-nat-gateway`

18. **Subnet ID**: The subnet in which the NAT gateway is deployed.
   - Example: `subnet-123abc45`

19. **Elastic IP Allocation ID**: The allocation ID of the Elastic IP address associated with the NAT gateway.
   - Example: `eipalloc-123abc45`

Security Groups

20. **Security Group ID**: The unique identifier for the security group.
   - Example: `sg-123abc45`

21. **Security Group Name**: The name assigned to the security group.
   - Example: `my-security-group`

22. **Inbound Rules**: The rules that control the inbound traffic to the security group.
   - Example: `[ { "IpProtocol": "tcp", "FromPort": 80, "ToPort": 80, "IpRanges": [ { "CidrIp": "0.0.0.0/0" } ] } ]`

23. **Outbound Rules**: The rules that control the outbound traffic from the security group.
   - Example: `[ { "IpProtocol": "tcp", "FromPort": 80, "ToPort": 80, "IpRanges": [ { "CidrIp": "0.0.0.0/0" } ] } ]`

Network ACLs (NACLs)

24. **Network ACL ID**: The unique identifier for the network ACL.

   - Example: `acl-123abc45`

25. **Network ACL Name**: The name assigned to the network ACL.

   - Example: `my-nacl`

26. **Inbound Rules**: The rules that control the inbound traffic to the network ACL.

   - Example: `[ { "RuleNumber": 100, "Protocol": "tcp", "RuleAction": "allow", "CidrBlock": "0.0.0.0/0", "PortRange": { "From": 80, "To": 80 } } ]`

27. **Outbound Rules**: The rules that control the outbound traffic from the network ACL.

   - Example: `[ { "RuleNumber": 100, "Protocol": "tcp", "RuleAction": "allow", "CidrBlock": "0.0.0.0/0", "PortRange": { "From": 80, "To": 80 } } ]`

DHCP Options

28. **DHCP Options Set ID**: The unique identifier for the DHCP options set.

   - Example: `dopt-123abc45`

29. **Domain Name**: The domain name to assign to instances in the VPC.

   - Example: `example.com`

30. **Domain Name Servers**: The IP addresses of domain name servers.

   - Example: `[ "10.0.0.2", "10.0.0.3" ]`

31. **NTP Servers**: The IP addresses of Network Time Protocol (NTP) servers.

   - Example: `[ "10.0.0.4", "10.0.0.5" ]`

32. **NetBIOS Name Servers**: The IP addresses of NetBIOS name servers.

   - Example: `[ "10.0.0.6", "10.0.0.7" ]`

33. **NetBIOS Node Type**: The NetBIOS node type (1, 2, 4, or 8).
   - Example: `2`

Peering Connections

34. **VPC Peering Connection ID**: The unique identifier for the VPC peering connection.
   - Example: `pcx-123abc45`

35. **Requester VPC ID**: The ID of the requester VPC.
   - Example: `vpc-123abc45`

36. **Accepter VPC ID**: The ID of the accepter VPC.
   - Example: `vpc-678def90`

Endpoints

37. **Endpoint ID**: The unique identifier for the VPC endpoint.
   - Example: `vpce-123abc45`

38. **Endpoint Type**: The type of endpoint (Interface, Gateway).
   - Example: `Interface`

39. **Service Name**: The name of the AWS service to which the endpoint connects.
   - Example: `com.amazonaws.us-west-2.s3`

40. **Subnet IDs**: The subnets in which the endpoint is available.
   - Example: `[ "subnet-123abc45", "subnet-678def90" ]`

Elastic IPs

41. **Elastic IP Allocation ID**: The allocation ID of the Elastic IP address.
   - Example: `eipalloc-123abc45`

42. **Public IP**: The public IP address associated with the Elastic IP.

   - Example: `203.0.113.0`

VPN Connections

43. **VPN Connection ID**: The unique identifier for the VPN connection.

   - Example: `vpn-123abc45`

44. **Customer Gateway ID**: The ID of the customer gateway.

   - Example: `cgw-123abc45`

45. **Virtual Private Gateway ID**: The ID of the virtual private gateway.

   - Example: `vgw-123abc45`

46. **Tunnel Options**: Configuration options for the VPN tunnels.

   - Example: `{ "OutsideIpAddress": "198.51.100.1", "PreSharedKey": "example-key", ... }`

Miscellaneous

47. **Flow Logs**: Captures detailed information about the IP traffic going to and from network interfaces in the VPC.

   - Example: `{ "LogGroupName": "/aws/vpc/flowlogs", "TrafficType": "ALL", "DeliverLogsPermissionArn": "arn:aws:iam::123456789012:role/flowlogsRole" }`

48. **Endpoints**: The network interface that serves as an entry point for traffic destined to a supported AWS service.

   - Example: `{ "VpcEndpointId": "vpce-123abc45", "VpcEndpointType": "Interface", ... }`

49. **Tags**: Key-value pairs that you can use to categorize and manage your VPC resources.

   - Example: `{ "Key": "Environment", "Value": "Production" }`

These parameters can be configured through the AWS Management Console, AWS CLI, or AWS SDKs, providing flexibility in managing your VPC resources.

---

6. List of all Direct Connect parameters and their definition and description

AWS Direct Connect is a cloud service solution that makes it easy to establish a dedicated network connection from your premises to AWS. Here is a comprehensive list of Direct Connect parameters, along with their definitions and descriptions:

General Parameters

1. **Connection ID**: The unique identifier for the Direct Connect connection.
   - Example: `dxcon-abcdef01`

2. **Connection Name**: The name assigned to the Direct Connect connection.
   - Example: `my-connection`

3. **Connection State**: The current state of the connection.
   - Example: `requested`, `pending`, `available`, `down`, `deleting`, `deleted`, `rejected`

4. **Location**: The AWS Direct Connect location where the connection is established.
   - Example: `EqDC2`

5. **Bandwidth**: The bandwidth of the connection.
   - Example: `1Gbps`, `10Gbps`

Virtual Interfaces (VIF)

6. **Virtual Interface ID**: The unique identifier for the virtual interface.
   - Example: `dxvif-abcdef01`

7. **Virtual Interface Name**: The name assigned to the virtual interface.
   - Example: `my-virtual-interface`

8. **Virtual Interface Type**: The type of virtual interface.

   - Example: `private`, `public`, `transit`

9. **Virtual Interface State**: The current state of the virtual interface.

   - Example: `confirming`, `verifying`, `pending`, `available`, `down`, `deleting`, `deleted`, `rejected`

10. **VLAN**: The VLAN ID for the virtual interface.

    - Example: `101`

11. **ASN**: The autonomous system number (ASN) for Border Gateway Protocol (BGP) configuration.

    - Example: `65000`

12. **Amazon Address**: The Amazon side IP address for the virtual interface.

    - Example: `192.0.2.1/30`

13. **Customer Address**: The customer side IP address for the virtual interface.

    - Example: `192.0.2.2/30`

14. **BGP Peer**: The BGP peer information.

    - Example: `{ "Asn": 65000, "AuthKey": "example-key", "AmazonAddress": "192.0.2.1", "CustomerAddress": "192.0.2.2" }`

Link Aggregation Group (LAG)

15. **LAG ID**: The unique identifier for the link aggregation group.

    - Example: `dxlag-abcdef01`

16. **LAG Name**: The name assigned to the link aggregation group.

    - Example: `my-lag`

17. **LAG State**: The current state of the link aggregation group.

   - Example: `requested`, `pending`, `available`, `down`, `deleting`, `deleted`

18. **Connections**: The connections that are part of the link aggregation group.

   - Example: `[ "dxcon-abcdef01", "dxcon-abcdef02" ]`

19. **Minimum Links**: The minimum number of physical connections that must be operational for the link aggregation group to be operational.

   - Example: `2`

Tags and Metadata

20. **Tags**: Key-value pairs that you can use to categorize and manage your Direct Connect resources.

   - Example: `{ "Key": "Environment", "Value": "Production" }`

21. **Owner Account**: The AWS account that owns the Direct Connect connection.

   - Example: `123456789012`

High Availability and Redundancy

22. **Redundant Connection ID**: The ID of the redundant connection.

   - Example: `dxcon-abcdef02`

23. **Redundant Connection State**: The state of the redundant connection.

   - Example: `available`

Configuration and Provisioning

24. **MTU**: The maximum transmission unit (MTU) for the connection.

   - Example: `1500`, `9001`

25. **Jumbo Frame Support**: Indicates whether jumbo frames are supported.

   - Example: `Enabled`

26. **AWS Device**: The AWS Direct Connect endpoint device.

   - Example: `aws-device-1`

27. **Partner Name**: The name of the AWS Direct Connect partner if the connection is provisioned through a partner.

   - Example: `Equinix`

Billing and Usage

28. **Billing Rate**: The billing rate for the Direct Connect connection.

   - Example: `hourly`

29. **Usage Data**: Information about the data usage for the Direct Connect connection.

   - Example: `{ "Inbound": "500GB", "Outbound": "600GB" }`

Monitoring and Alerts

30. **CloudWatch Metrics**: Metrics available in Amazon CloudWatch for monitoring the Direct Connect connection.

   - Example: `{ "MetricName": "BytesIn", "Namespace": "AWS/DirectConnect", "Statistic": "Sum", "Period": 300 }`

31. **Notifications**: Configuration for receiving notifications about the connection status.

   - Example: `{ "Email": "admin@example.com", "SNSTopic": "arn:aws:sns:us-west-2:123456789012:MyTopic" }`

Miscellaneous

32. **LOA-CFA**: The Letter of Authorization and Connecting Facility Assignment document required for setting up the connection.

- Example: `loa-cfa-abcdef01`

33. **Connection Mode**: The connection mode (standard or hosted).

- Example: `standard`

These parameters can be configured and managed through the AWS Management Console, AWS CLI, or AWS SDKs, providing flexibility in setting up and maintaining your Direct Connect connections.

---

7. List of all Lambda parameters and their definition and description

Amazon Lambda allows you to run code without provisioning or managing servers. Here is a comprehensive list of Lambda function parameters, along with their definitions and descriptions:

Basic Parameters

1. **Function Name**: The name of the Lambda function. It must be unique within an AWS Region.

- Example: `my-lambda-function`

2. **Function ARN**: The Amazon Resource Name (ARN) that uniquely identifies the Lambda function.

- Example: `arn:aws:lambda:us-west-2:123456789012:function:my-lambda-function`

3. **Runtime**: The runtime environment for the Lambda function. AWS provides various runtimes such as Node.js, Python, Java, etc.

- Example: `nodejs14.x`, `python3.8`

4. **Handler**: The method within your code that Lambda calls to execute the function. It typically includes the file name and the function name.

- Example: `index.handler`

5. **Role**: The ARN of the IAM role that Lambda assumes when it executes your function.

- Example: `arn:aws:iam::123456789012:role/execution_role`

6. **Description**: A brief description of the Lambda function.

   - Example: `This function processes user data.`

Code Parameters

7. **Code Location**: The location of the function code, either an S3 bucket or an inline zip file.

   - Example: `{ "S3Bucket": "my-bucket", "S3Key": "my-function.zip" }`

8. **Code Size**: The size of the function code in bytes.

   - Example: `1234567`

Configuration Parameters

9. **Timeout**: The maximum execution time for the Lambda function, in seconds. The default is 3 seconds.

   - Example: `30`

10. **Memory Size**: The amount of memory available to the function at runtime. Increasing the memory also increases the CPU allocation.

    - Example: `128`, `256`, `512`

11. **Environment Variables**: Key-value pairs that Lambda passes to your function.

    - Example: `{ "KEY": "value" }`

12. **Dead Letter Queue (DLQ)**: The ARN of an SQS queue or SNS topic where Lambda sends events that it can't process.

    - Example: `arn:aws:sqs:us-west-2:123456789012:my-queue`

13. **KMS Key**: The ARN of the KMS key used to encrypt environment variables.

    - Example: `arn:aws:kms:us-west-2:123456789012:key/abcd-1234-efgh-5678`

14. **File System Config**: Configuration for connecting to an Amazon EFS file system.

- Example: `{ "Arn": "arn:aws:elasticfilesystem:us-west-2:123456789012:access-point/fsap-0123456789abcdef0", "LocalMountPath": "/mnt/efs" }`

Networking Parameters

15. **VPC Configuration**: The VPC subnets and security groups that Lambda uses to set up an elastic network interface.

- Example: `{ "SubnetIds": [ "subnet-abcde123" ], "SecurityGroupIds": [ "sg-123abcde" ] }`

Triggers and Event Sources

16. **Event Source Mappings**: Configurations that specify how Lambda should read from event sources such as DynamoDB streams, Kinesis streams, or SQS queues.

- Example: `{ "EventSourceArn": "arn:aws:dynamodb:us-west-2:123456789012:table/my-table/stream/2020-01-01T00:00:00.000", "BatchSize": 100 }`

17. **Triggers**: Configurations that specify which services can trigger the Lambda function.

- Example: `{ "Type": "API Gateway", "Arn": "arn:aws:apigateway:us-west-2::/restapis/a1b2c3d4e5/prod/POST/myresource" }`

Tags and Metadata

18. **Tags**: Key-value pairs that you can use to categorize and manage your Lambda functions.

- Example: `{ "Environment": "Production", "Department": "IT" }`

19. **Revision ID**: A unique identifier for the current version of the function code and configuration.

- Example: `e4c8bcff-9e07-4a74-a98d-345123456789`

Versions and Aliases

20. **Version**: The version number of the Lambda function.

- Example: `1`, `2`, `$LATEST`

21. **Alias**: A pointer to a specific version of the Lambda function, allowing you to manage different environments (e.g., development, production).

   - Example: `{ "Name": "prod", "FunctionVersion": "2", "Description": "Production environment" }`

Monitoring and Logging

22. **CloudWatch Logs**: The log group where Lambda sends execution logs.

   - Example: `/aws/lambda/my-lambda-function`

23. **Tracing**: The tracing mode that Lambda uses to trace requests with AWS X-Ray.

   - Example: `Active`, `PassThrough`

Reserved and Provisioned Concurrency

24. **Reserved Concurrency**: The maximum number of concurrent instances of the function that can be reserved.

   - Example: `10`

25. **Provisioned Concurrency**: The number of concurrent instances of the function that are pre-provisioned and ready to respond to requests.

   - Example: `5`

Miscellaneous

26. **Last Modified**: The date and time when the function was last updated.

   - Example: `2021-01-01T12:00:00.000+0000`

27. **State**: The current state of the Lambda function.

   - Example: `Active`, `Inactive`, `Pending`

28. **State Reason**: The reason for the current state of the Lambda function.

   - Example: `The function is being created.`

29. **State Reason Code**: The code associated with the reason for the function's current state.

   - Example: `Creating`

30. **Layers**: A list of Lambda layers that the function can use.

   - Example: `[ "arn:aws:lambda:us-west-2:123456789012:layer:my-layer:1" ]`

31. **Ephemeral Storage**: The amount of ephemeral storage (/tmp) available for the function in MB.

   - Example: `512`

These parameters can be configured and managed through the AWS Management Console, AWS CLI, or AWS SDKs, providing flexibility in managing your Lambda functions.

---

8. List of all API Gateway parameters and their definition and description

Amazon API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs. Here is a comprehensive list of API Gateway parameters, along with their definitions and descriptions:

General API Parameters

1. **API ID**: The unique identifier for the API.

   - Example: `a1b2c3d4`

2. **API Name**: The name assigned to the API.

   - Example: `MyAPI`

3. **API Description**: A brief description of the API.

   - Example: `This API handles user authentication.`

4. **API Version**: The version identifier for the API.

   - Example: `v1`

5. **API Endpoint Type**: The endpoint type of the API. Options include `REGIONAL`, `EDGE`, and `PRIVATE`.

   - Example: `REGIONAL`

Resources and Methods

6. **Resource ID**: The unique identifier for a resource within the API.

   - Example: `a1b2c3`

7. **Resource Path**: The path of the resource within the API.

   - Example: `/users`

8. **HTTP Method**: The HTTP method for the resource (e.g., GET, POST, PUT, DELETE).

   - Example: `GET`

9. **Method Request Parameters**: Parameters required by the method request.

   - Example: `{ "method.request.querystring.name": "true" }`

10. **Method Request Models**: Models that define the structure of the request payload.

    - Example: `{ "application/json": "UserModel" }`

11. **Method Response Parameters**: Parameters returned in the method response.

    - Example: `{ "method.response.header.Content-Type": "true" }`

12. **Method Response Models**: Models that define the structure of the response payload.

    - Example: `{ "application/json": "UserResponseModel" }`

Integrations

13. **Integration Type**: The type of backend integration. Options include `HTTP`, `AWS`, `MOCK`, `HTTP_PROXY`, and `AWS_PROXY`.

    - Example: `AWS_PROXY`

14. **Integration URI**: The URI of the integration endpoint.

   - Example: `arn:aws:lambda:us-west-2:123456789012:function:my-function`

15. **Integration HTTP Method**: The HTTP method for the integration request.

   - Example: `POST`

16. **Integration Request Parameters**: Parameters sent to the integration endpoint.

   - Example: `{ "integration.request.querystring.name": "method.request.querystring.name" }`

17. **Integration Request Templates**: Mapping templates that transform the incoming request data.

   - Example: `{ "application/json": "{ \"name\": \"$input.params('name')\" }" }`

18. **Integration Response Parameters**: Parameters returned from the integration endpoint.

   - Example: `{ "integration.response.header.Content-Type": "method.response.header.Content-Type" }`

19. **Integration Response Templates**: Mapping templates that transform the response data.

   - Example: `{ "application/json": "{ \"message\": \"Hello, $input.path('$.name')\" }" }`

Stages

20. **Stage Name**: The name of the stage.

   - Example: `prod`

21. **Deployment ID**: The unique identifier for the deployment.

   - Example: `a1b2c3`

22. **Stage Variables**: Key-value pairs that you can use to customize the stage configuration.

   - Example: `{ "LambdaAlias": "prod" }`

23. **Cache Cluster Enabled**: Indicates whether a cache cluster is enabled for the stage.
   - Example: `true`, `false`

24. **Cache Cluster Size**: The size of the cache cluster.
   - Example: `0.5`, `1.6`

25. **Logging Level**: The logging level for the stage. Options include `OFF`, `ERROR`, and `INFO`.
   - Example: `INFO`

26. **Data Trace Enabled**: Indicates whether data trace logging is enabled for the stage.
   - Example: `true`, `false`

Authorizers

27. **Authorizer ID**: The unique identifier for the authorizer.
   - Example: `a1b2c3`

28. **Authorizer Name**: The name of the authorizer.
   - Example: `MyAuthorizer`

29. **Authorizer Type**: The type of authorizer. Options include `TOKEN`, `REQUEST`, and `COGNITO_USER_POOLS`.
   - Example: `TOKEN`

30. **Authorizer URI**: The URI of the Lambda function used as the authorizer.
   - Example: `arn:aws:lambda:us-west-2:123456789012:function:my-authorizer`

31. **Authorizer Credentials**: The ARN of the IAM role that the authorizer uses.
   - Example: `arn:aws:iam::123456789012:role/my-role`

Throttling and Quotas

32. **Throttle Rate Limit**: The maximum rate of requests per second.
   - Example: `1000`

33. **Throttle Burst Limit**: The maximum burst of requests.
   - Example: `2000`

34. **Quota Limit**: The maximum number of requests per day, week, or month.
   - Example: `{ "limit": 10000, "period": "DAY" }`

CORS Configuration

35. **CORS Enabled**: Indicates whether CORS (Cross-Origin Resource Sharing) is enabled.
   - Example: `true`, `false`

36. **Allowed Origins**: The origins that are allowed to access the API.
   - Example: `["*"]`

37. **Allowed Methods**: The HTTP methods that are allowed.
   - Example: `["GET", "POST"]`

38. **Allowed Headers**: The headers that are allowed.
   - Example: `["Content-Type"]`

39. **Exposed Headers**: The headers that are exposed.
   - Example: `["X-My-Header"]`

40. **Max Age**: The maximum age for the CORS preflight request cache.
   - Example: `3600`

Monitoring and Logging

41. **CloudWatch Metrics Enabled**: Indicates whether CloudWatch metrics are enabled.

- Example: `true`, `false`

42. **Access Logging Enabled**: Indicates whether access logging is enabled.
    - Example: `true`, `false`

43. **Access Log Destination ARN**: The ARN of the destination for access logs.
    - Example: `arn:aws:logs:us-west-2:123456789012:log-group:/aws/apigateway/my-access-logs`

44. **Access Log Format**: The format of the access logs.
    - Example: `$context.requestId $context.identity.sourceIp $context.requestTime`

Miscellaneous

45. **API Key Source**: The source of the API keys. Options include `HEADER` and `AUTHORIZER`.
    - Example: `HEADER`

46. **Endpoint Configuration Types**: Types of endpoint configurations. Options include `REGIONAL`, `EDGE`, `PRIVATE`.
    - Example: `REGIONAL`

47. **Binary Media Types**: A list of binary media types supported by the API.
    - Example: `["image/png", "application/octet-stream"]`

48. **Tags**: Key-value pairs that you can use to categorize and manage your API Gateway resources.
    - Example: `{ "Environment": "Production", "Department": "IT" }`

These parameters can be configured and managed through the AWS Management Console, AWS CLI, or AWS SDKs, providing flexibility in setting up and maintaining your API Gateway resources.

---

9. List of all ECS parameters and their definition and description

Amazon Elastic Container Service (ECS) is a highly scalable, high-performance container orchestration service that supports Docker containers and allows you to run and manage containerized applications. Below is a comprehensive list of ECS instance parameters along with their definitions and descriptions:

Cluster Parameters

1. **Cluster Name**: The name of the ECS cluster.
   - Example: `my-cluster`

2. **Cluster ARN**: The Amazon Resource Name (ARN) of the ECS cluster.
   - Example: `arn:aws:ecs:us-west-2:123456789012:cluster/my-cluster`

3. **Cluster Status**: The current status of the ECS cluster.
   - Example: `ACTIVE`, `PROVISIONING`, `DEPROVISIONING`, `FAILED`

Task Definition Parameters

4. **Task Definition Name**: The name of the task definition.
   - Example: `my-task`

5. **Task Definition ARN**: The ARN of the task definition.
   - Example: `arn:aws:ecs:us-west-2:123456789012:task-definition/my-task:1`

6. **Task Definition Family**: The family of the task definition.
   - Example: `my-task-family`

7. **Task Definition Revision**: The revision number of the task definition.
   - Example: `1`

8. **Container Definitions**: The container definitions within the task definition, including container image, memory, CPU, and port mappings.

- Example: `[ { "name": "web", "image": "nginx", "cpu": 256, "memory": 512, "portMappings": [ { "containerPort": 80, "hostPort": 80 } ] } ]`

9. **Task Role ARN**: The ARN of the IAM role that containers in the task can assume.
   - Example: `arn:aws:iam::123456789012:role/ecsTaskExecutionRole`

10. **Network Mode**: The Docker networking mode for the containers in the task.
    - Example: `bridge`, `host`, `awsvpc`, `none`

11. **Volumes**: The volumes that the containers in the task can use.
    - Example: `[ { "name": "my-volume", "host": { "sourcePath": "/ecs/data" } } ]`

12. **Requires Compatibilities**: The launch types the task requires.
    - Example: `[ "EC2", "FARGATE" ]`

Service Parameters

13. **Service Name**: The name of the ECS service.
    - Example: `my-service`

14. **Service ARN**: The ARN of the ECS service.
    - Example: `arn:aws:ecs:us-west-2:123456789012:service/my-service`

15. **Service Status**: The current status of the ECS service.
    - Example: `ACTIVE`, `DRAINING`, `INACTIVE`

16. **Desired Count**: The desired number of tasks to run in the service.
    - Example: `3`

17. **Running Count**: The number of tasks that are currently running.
    - Example: `2`

18. **Pending Count**: The number of tasks that are currently pending.
   - Example: `1`

19. **Launch Type**: The launch type for the service.
   - Example: `EC2`, `FARGATE`

20. **Cluster**: The name of the ECS cluster that hosts the service.
   - Example: `my-cluster`

Container Instance Parameters

21. **Container Instance ID**: The ID of the container instance.
   - Example: `arn:aws:ecs:us-west-2:123456789012:container-instance/abcdef01`

22. **Container Instance Status**: The current status of the container instance.
   - Example: `ACTIVE`, `DRAINING`, `REGISTERING`, `DEREGISTERING`, `REGISTRATION_FAILED`

23. **EC2 Instance ID**: The ID of the EC2 instance associated with the container instance.
   - Example: `i-1234567890abcdef0`

24. **Agent Connected**: Indicates whether the Amazon ECS container agent is connected to the instance.
   - Example: `true`, `false`

25. **Running Tasks Count**: The number of tasks running on the container instance.
   - Example: `5`

26. **Pending Tasks Count**: The number of tasks pending on the container instance.
   - Example: `2`

27. **Registered Resources**: The resources registered to the container instance (e.g., CPU, memory).

- Example: `[ { "name": "CPU", "type": "INTEGER", "doubleValue": 0.0, "longValue": 0, "integerValue": 2048, "stringSetValue": null }, { "name": "MEMORY", "type": "INTEGER", "doubleValue": 0.0, "longValue": 0, "integerValue": 8192, "stringSetValue": null } ]`

28. **Remaining Resources**: The remaining resources available on the container instance.

- Example: `[ { "name": "CPU", "type": "INTEGER", "doubleValue": 0.0, "longValue": 0, "integerValue": 1024, "stringSetValue": null }, { "name": "MEMORY", "type": "INTEGER", "doubleValue": 0.0, "longValue": 0, "integerValue": 4096, "stringSetValue": null } ]`

Load Balancing Parameters

29. **Load Balancers**: The load balancers associated with the service.

- Example: `[ { "targetGroupArn": "arn:aws:elasticloadbalancing:us-west-2:123456789012:targetgroup/my-target-group/50dc6c495c0c9188", "containerName": "web", "containerPort": 80 } ]`

30. **Service Registries**: The service registries for the service.

- Example: `[ { "registryArn": "arn:aws:servicediscovery:us-west-2:123456789012:service/srv-e4anhexample0004", "port": 80, "containerName": "web", "containerPort": 80 } ]`

Autoscaling Parameters

31. **Scaling Policies**: The scaling policies associated with the service.

- Example: `[ { "policyName": "my-scaling-policy", "targetTrackingScalingPolicyConfiguration": { "targetValue": 75.0, "predefinedMetricSpecification": { "predefinedMetricType": "ECSServiceAverageCPUUtilization" } } } ]`

Network Configuration

32. **AwsvpcConfiguration**: The network configuration for tasks that use the `awsvpc` network mode.

- Example: `{ "subnets": [ "subnet-12345678" ], "securityGroups": [ "sg-12345678" ], "assignPublicIp": "ENABLED" }`

Logging Parameters

33. **Log Configuration**: The log configuration for the container.

   - Example: `{ "logDriver": "awslogs", "options": { "awslogs-group": "/ecs/my-service", "awslogs-region": "us-west-2", "awslogs-stream-prefix": "ecs" } }`

Tagging and Metadata

34. **Tags**: Key-value pairs that you can use to categorize and manage your ECS resources.

   - Example: `{ "Key": "Environment", "Value": "Production" }`

35. **CreatedAt**: The timestamp when the resource was created.

   - Example: `2021-01-01T12:00:00.000Z`

36. **UpdatedAt**: The timestamp when the resource was last updated.

   - Example: `2021-01-02T12:00:00.000Z`

Task Parameters

37. **Task ID**: The ID of the task.

   - Example: `arn:aws:ecs:us-west-2:123456789012:task/a1b2c3d4e5`

38. **Task Status**: The current status of the task.

   - Example: `RUNNING`, `PENDING`, `STOPPED`

39. **Task Desired Status**: The desired status of the task.

   - Example: `RUNNING`, `STOPPED`

40. **Task Definition ARN**: The ARN of the task definition that the task is using.

   - Example: `arn:aws:ecs:us-west-2:123456789012:task-definition/my-task:1`

These parameters can be configured and managed through the AWS Management Console, AWS CLI, or AWS SDKs, providing flexibility in setting up and maintaining your ECS resources.

---

10. List of all EKS parameters and their definition and description

Amazon Elastic Kubernetes Service (EKS) is a managed service that makes it easy to run Kubernetes on AWS without needing to install and operate your own Kubernetes control plane or nodes. Below is a comprehensive list of EKS instance parameters along with their definitions and descriptions:

Cluster Parameters

1. **Cluster Name**: The name of the EKS cluster.
   - Example: `my-cluster`

2. **Cluster ARN**: The Amazon Resource Name (ARN) of the EKS cluster.
   - Example: `arn:aws:eks:us-west-2:123456789012:cluster/my-cluster`

3. **Kubernetes Version**: The Kubernetes version for the cluster.
   - Example: `1.21`

4. **Endpoint**: The endpoint URL for the EKS cluster's Kubernetes API server.
   - Example: `https://1234567890ABCDEF.gr7.us-west-2.eks.amazonaws.com`

5. **Role ARN**: The ARN of the IAM role that provides permissions for the EKS control plane to make calls to AWS API operations on your behalf.
   - Example: `arn:aws:iam::123456789012:role/EKS-ClusterRole`

6. **VPC Configuration**: The configuration of the VPC for the cluster.
   - Example: `{ "subnetIds": ["subnet-abcde123"], "securityGroupIds": ["sg-123abcde"] }`

7. **Logging Configuration**: The logging configuration for the cluster.
   - Example: `{ "clusterLogging": [ { "types": ["api", "audit", "authenticator"], "enabled": true } ] }`

8. **Certificate Authority Data**: The certificate authority data for the cluster's Kubernetes API server.

- Example: `{ "data": "LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCg==" }`

9. **Endpoint Public Access**: Indicates whether the cluster endpoint is publicly accessible.

- Example: `true`, `false`

10. **Endpoint Private Access**: Indicates whether the cluster endpoint is accessible from within the VPC.

- Example: `true`, `false`

11. **Tags**: Key-value pairs used to categorize and manage the cluster.

- Example: `{ "Environment": "Production", "Department": "IT" }`

12. **Encryption Configuration**: The encryption configuration for the cluster.

- Example: `{ "resources": ["secrets"], "provider": { "keyArn": "arn:aws:kms:us-west-2:123456789012:key/abcd-1234-efgh-5678" } }`

Node Group Parameters

13. **Node Group Name**: The name of the node group.

- Example: `my-node-group`

14. **Node Group ARN**: The ARN of the node group.

- Example: `arn:aws:eks:us-west-2:123456789012:nodegroup/my-cluster/my-node-group/abcd1234`

15. **Instance Types**: The EC2 instance types for the nodes in the node group.

- Example: `[ "m5.large", "t3.medium" ]`

16. **Scaling Configuration**: The scaling configuration for the node group.

- Example: `{ "minSize": 2, "maxSize": 5, "desiredSize": 3 }`

17. **AMI Type**: The Amazon Machine Image (AMI) type for the nodes.

- Example: `AL2_x86_64`, `AL2_x86_64_GPU`, `CUSTOM`

18. **Node Role**: The ARN of the IAM role for the nodes in the node group.
   - Example: `arn:aws:iam::123456789012:role/EKS-NodeRole`

19. **Subnets**: The subnets used by the node group.
   - Example: `[ "subnet-abcde123", "subnet-123abcde" ]`

20. **Remote Access Configuration**: The remote access configuration for the nodes in the node group.
   - Example: `{ "ec2SshKey": "my-keypair", "sourceSecurityGroups": [ "sg-123abcde" ] }`

21. **Labels**: Kubernetes labels applied to nodes in the node group.
   - Example: `{ "role": "worker" }`

22. **Taints**: Kubernetes taints applied to nodes in the node group.
   - Example: `{ "key": "role", "value": "worker", "effect": "NoSchedule" }`

23. **Update Configurations**: The update configuration for the node group.
   - Example: `{ "maxUnavailable": 1, "maxUnavailablePercentage": 10 }`

24. **Launch Template**: The launch template for the node group.
   - Example: `{ "id": "lt-0123456789abcdef0", "version": "1" }`

Add-on Parameters

25. **Add-on Name**: The name of the add-on.
   - Example: `vpc-cni`

26. **Add-on ARN**: The ARN of the add-on.
   - Example: `arn:aws:eks:us-west-2:123456789012:addon/my-cluster/vpc-cni/abcd1234`

27. **Add-on Version**: The version of the add-on.

- Example: `v1.7.5-eksbuild.1`

28. **Service Account Role ARN**: The ARN of the IAM role for the add-on's service account.
    - Example: `arn:aws:iam::123456789012:role/EKS-AddonRole`

29. **Configuration Values**: Configuration values for the add-on.
    - Example: `{ "env": { "ENABLE_POD_ENI": "true" } }`

Fargate Profile Parameters

30. **Fargate Profile Name**: The name of the Fargate profile.
    - Example: `my-fargate-profile`

31. **Fargate Profile ARN**: The ARN of the Fargate profile.
    - Example: `arn:aws:eks:us-west-2:123456789012:fargateprofile/my-cluster/my-fargate-profile/abcd1234`

32. **Pod Execution Role ARN**: The ARN of the IAM role for the Fargate profile.
    - Example: `arn:aws:iam::123456789012:role/EKS-FargatePodExecutionRole`

33. **Subnets**: The subnets used by the Fargate profile.
    - Example: `[ "subnet-abcde123", "subnet-123abcde" ]`

34. **Selectors**: The selectors for the Fargate profile.
    - Example: `[ { "namespace": "default", "labels": { "role": "fargate" } } ]`

Security Group Parameters

35. **Security Group IDs**: The security group IDs for the cluster's control plane.
    - Example: `[ "sg-123abcde" ]`

36. **Cluster Security Group ID**: The ID of the security group for the cluster's control plane.

- Example: `sg-123abcde`

Monitoring Parameters

37. **CloudWatch Logging**: The CloudWatch logging configuration for the cluster.
    - Example: `{ "clusterLogging": [ { "types": ["api", "audit", "authenticator"], "enabled": true } ] }`

38. **Prometheus Monitoring**: The configuration for Prometheus monitoring.
    - Example: `{ "enabled": true, "configMapName": "prometheus-config" }`

Miscellaneous

39. **Platform Version**: The EKS platform version for the cluster.
    - Example: `eks.3`

40. **CreatedAt**: The timestamp when the cluster was created.
    - Example: `2021-01-01T12:00:00.000Z`

41. **UpdatedAt**: The timestamp when the cluster was last updated.
    - Example: `2021-01-02T12:00:00.000Z`

42. **Status**: The current status of the cluster.
    - Example: `ACTIVE`, `CREATING`, `DELETING`, `FAILED`, `UPDATING`

These parameters can be configured and managed through the AWS Management Console, AWS CLI, or AWS SDKs, providing flexibility in setting up and maintaining your EKS resources.

---

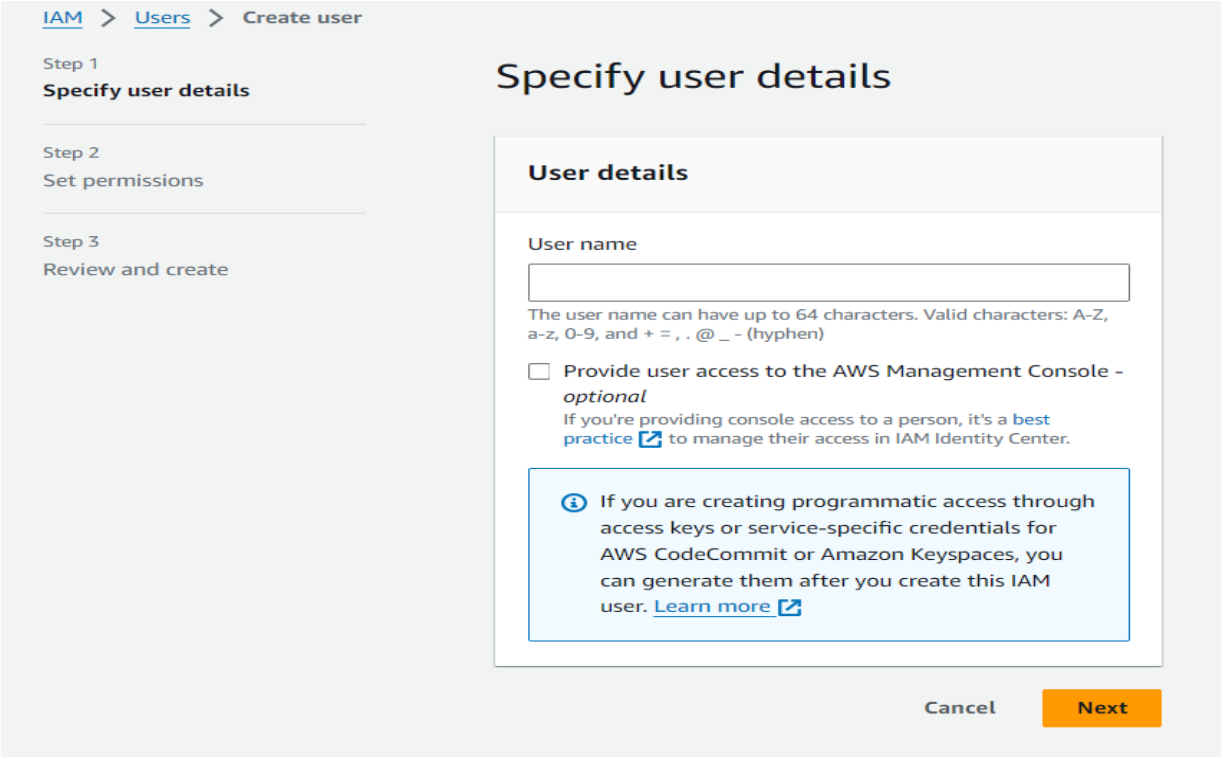
11. List of all IAM parameters and their definition and description

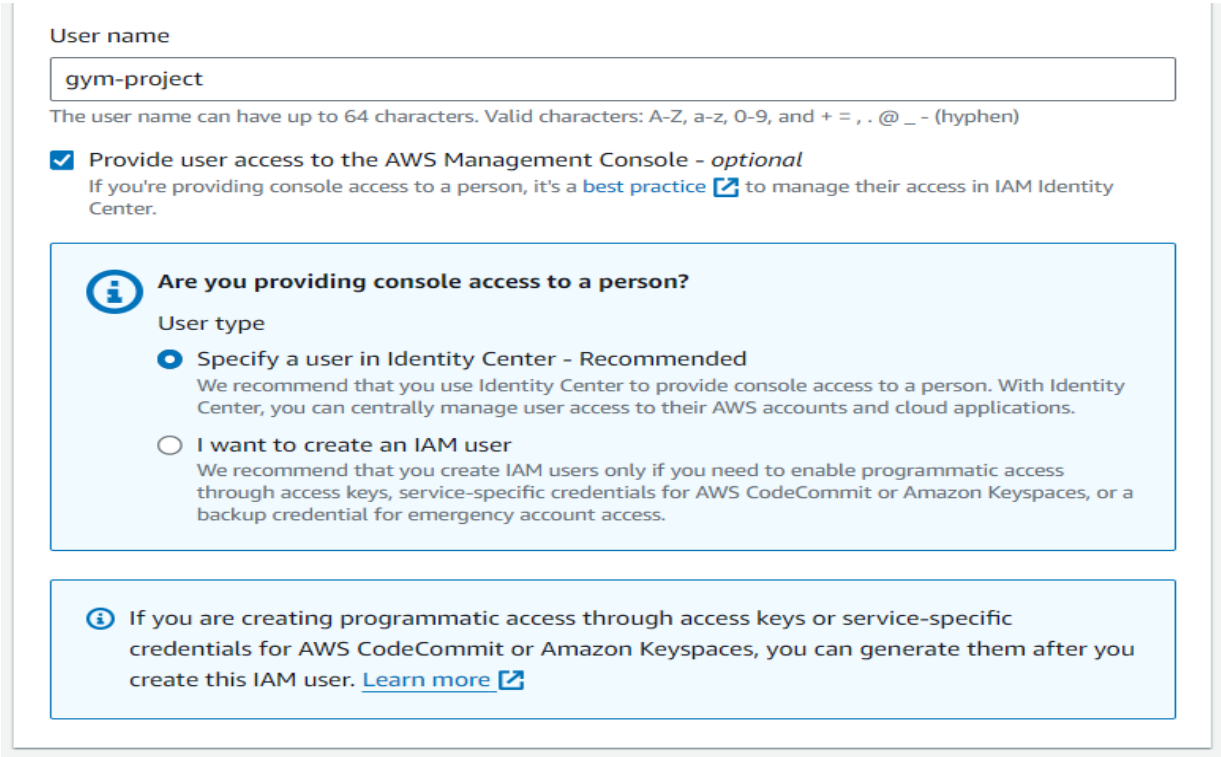
IAM (Identity and Access Management) in AWS provides fine-grained access control to AWS services and resources. Below is a comprehensive list of IAM parameters along with their definitions and descriptions:

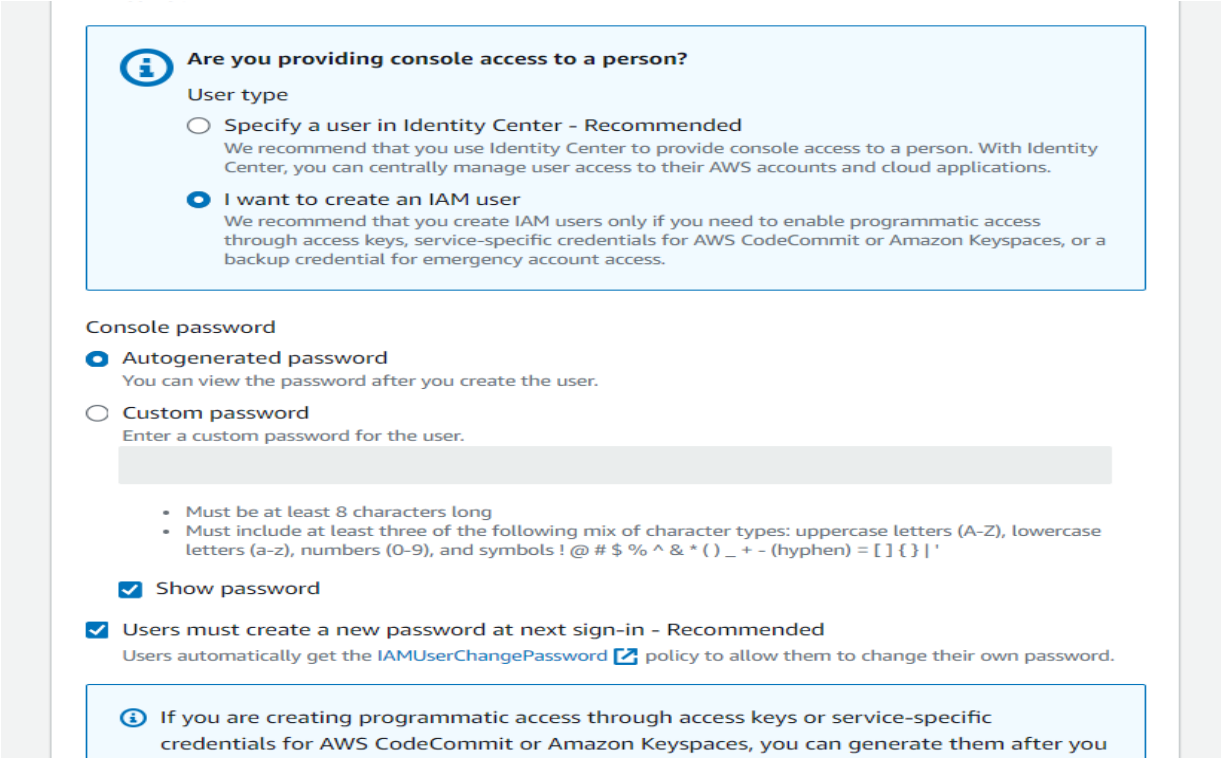
Users

1. **User Name**: The name of the IAM user.

   - Example: `john.doe`

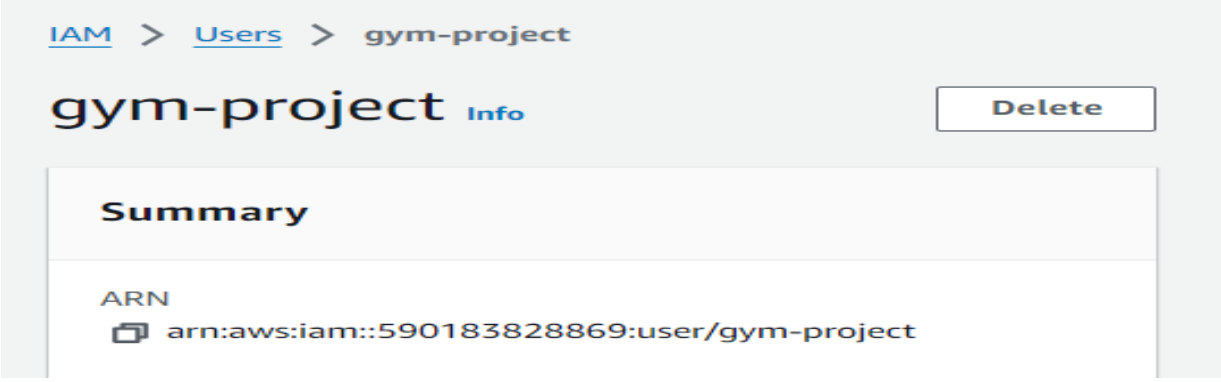






2. **User ARN**: The Amazon Resource Name (ARN) of the IAM user.

   - Example: `arn:aws:iam::123456789012:user/john.doe`



3. **User ID**: The unique identifier for the IAM user.

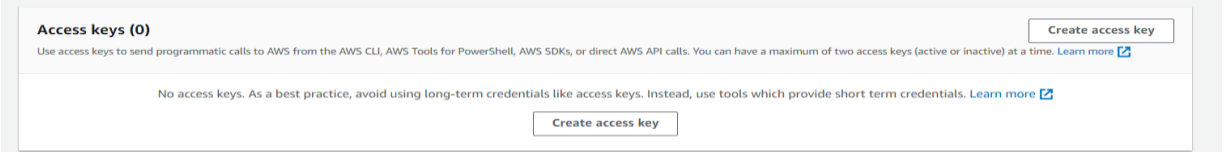
   - Example: `AIDA1B2C3D4E5`

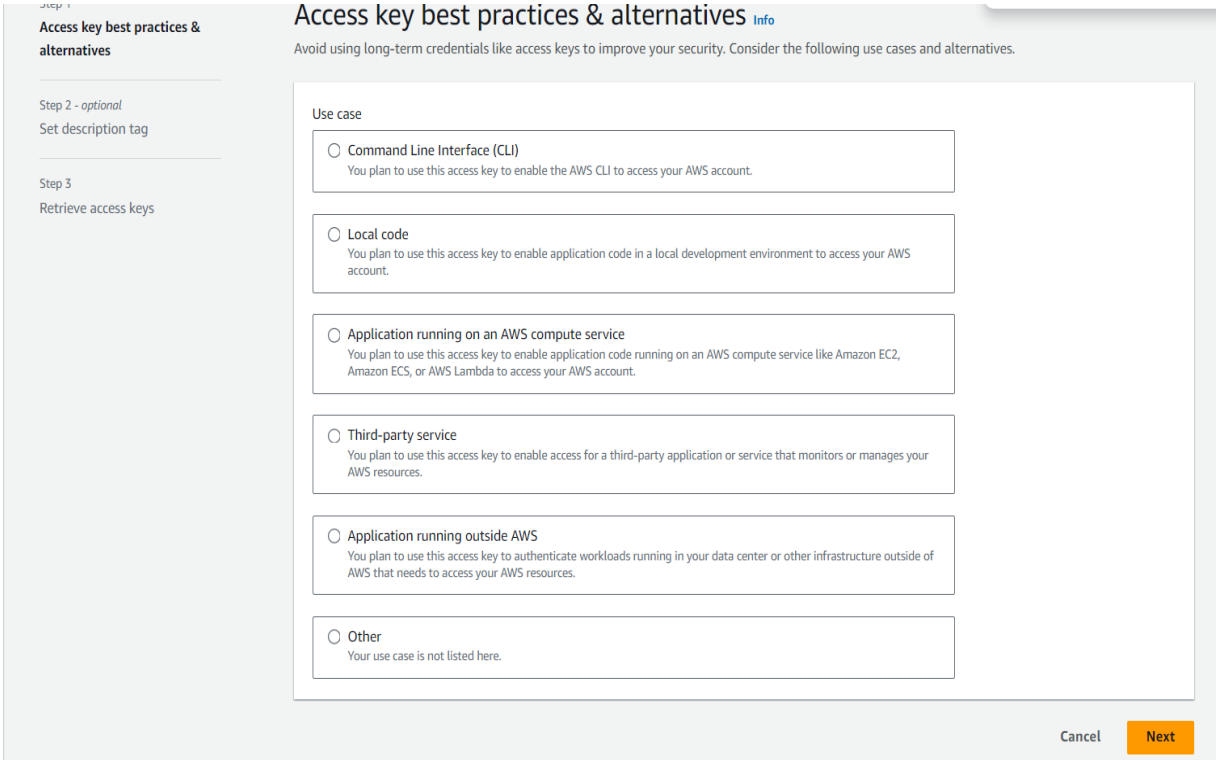
4. **Password Last Used**: The timestamp when the IAM user's password was last used.

   - Example: `2023-06-15T12:00:00Z`

5. **Access Key**: The access key ID for programmatic access.

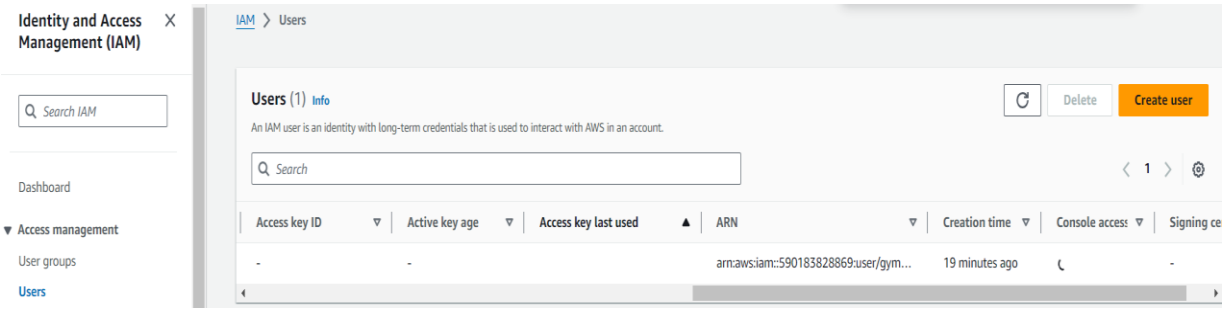
- Example: `AKIAIOSFODNN7EXAMPLE`





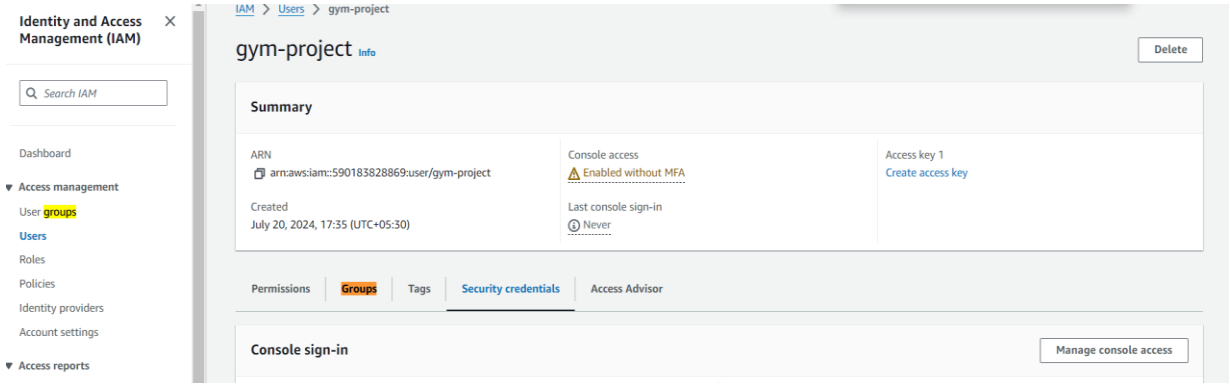
6. **Access Key Last Used**: Details about when the IAM access key was last used.

- Example: `{ "LastUsedDate": "2023-06-15T12:00:00Z", "ServiceName": "s3.amazonaws.com" }`



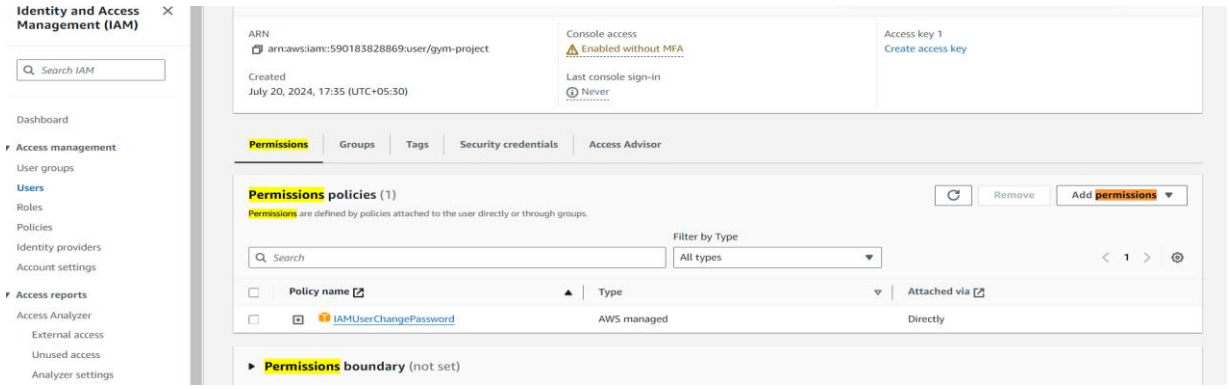
7. **Groups**: The IAM groups to which the user belongs.

   - Example: `[ "Admins", "Developers" ]`



8. **Permissions**: The permissions (policies) attached to the user.

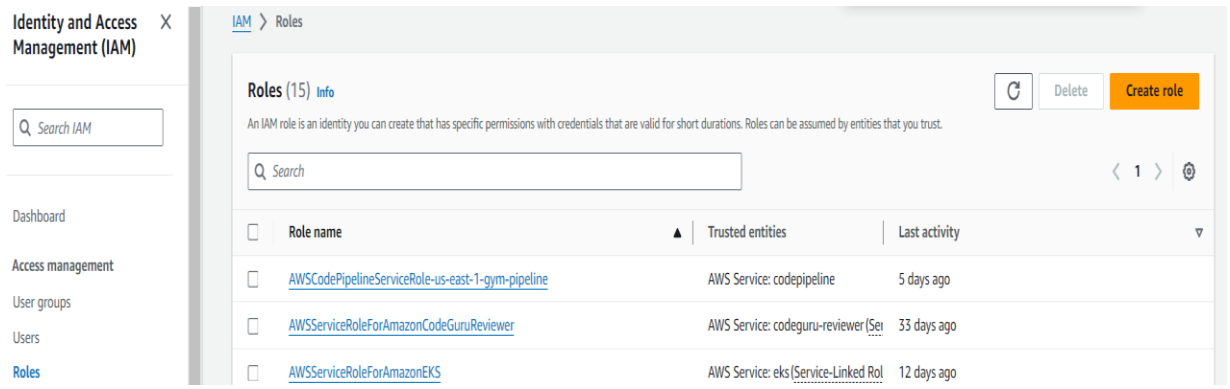
   - Example: `{ "Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Action": "*", "Resource": "*" } ] }`

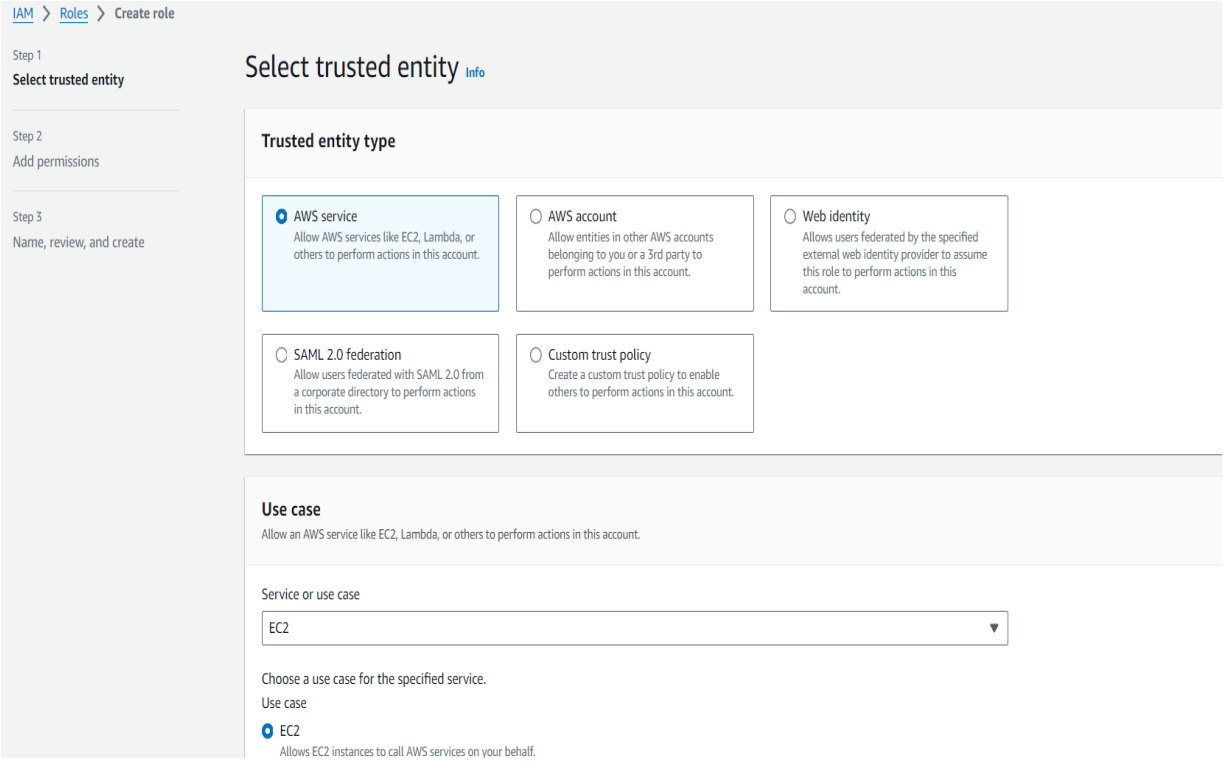


Roles

9. **Role Name**: The name of the IAM role.

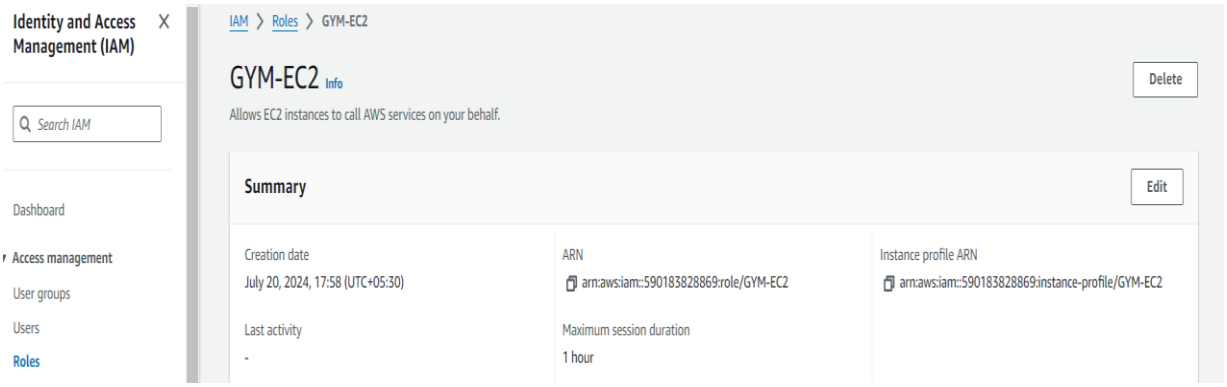
   - Example: `MyEC2Role`





10. **Role ARN**: The ARN of the IAM role.

   - Example: `arn:aws:iam::123456789012:role/MyEC2Role`



11. **Role ID**: The unique identifier for the IAM role.

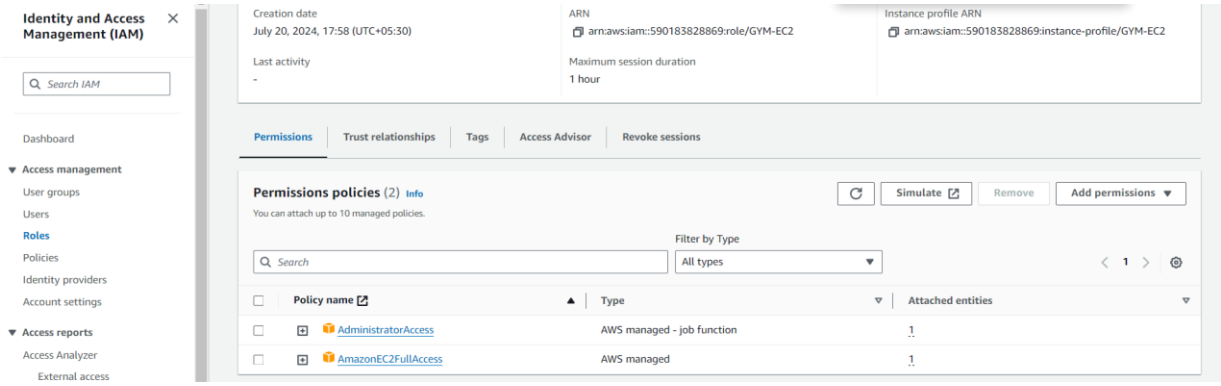
   - Example: `AIDAROLEID123`

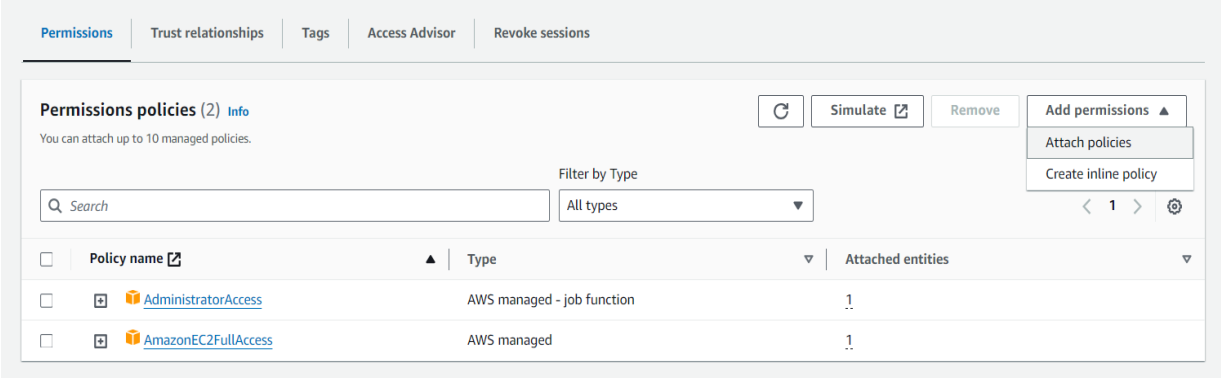
12. **Assume Role Policy**: The policy that defines who can assume the role.

   - Example: `{ "Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Principal": { "Service": "ec2.amazonaws.com" }, "Action": "sts:AssumeRole" } ] }`

13. **Attached Policies**: The policies attached to the IAM role.

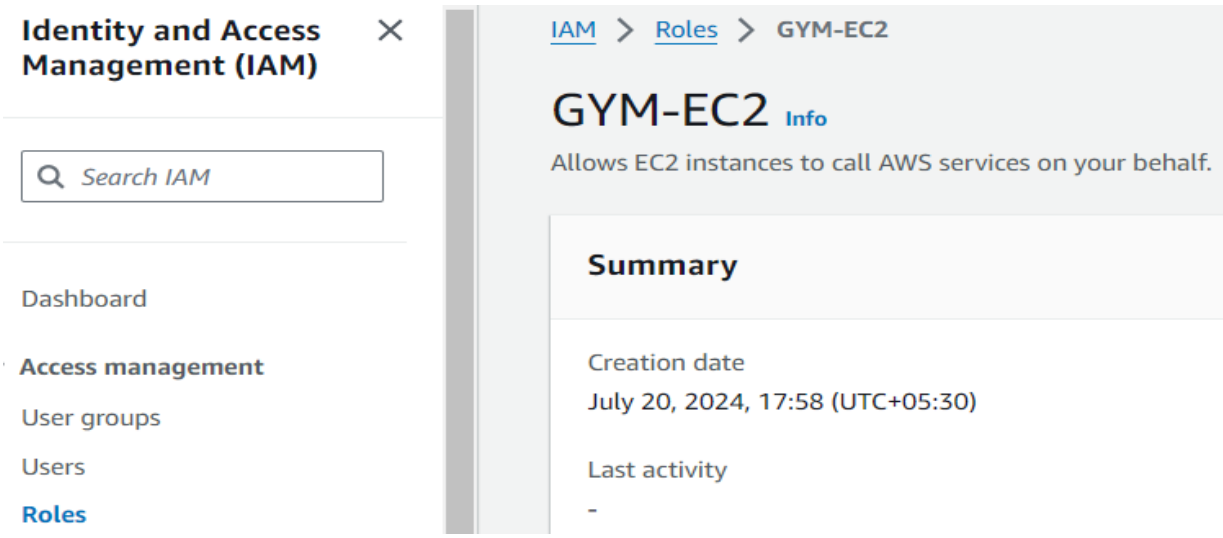
   - Example: `[ "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess" ]`





14. **Role Creation Date**: The timestamp when the IAM role was created.

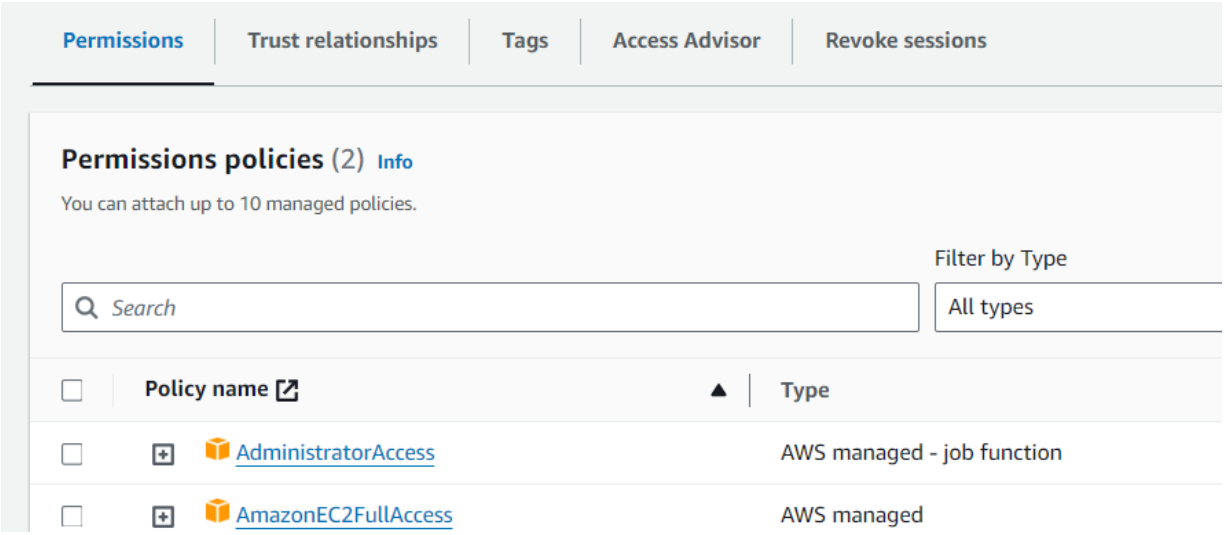
- Example: `2023-01-01T12:00:00Z`



Policies

15. **Policy Name**: The name of the IAM policy.

   - Example: `S3ReadOnlyAccess`



16. **Policy ARN**: The ARN of the IAM policy.

   - Example: `arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess`

17. **Policy ID**: The unique identifier for the IAM policy.

   - Example: `POLICYID123`

18. **Policy Document**: The JSON policy document that specifies permissions.

   - Example: `{ "Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Action": "s3:Get*", "Resource": "arn:aws:s3:::my-bucket/*" } ] }`

Groups

19. **Group Name**: The name of the IAM group.

   - Example: `Admins`

20. **Group ARN**: The ARN of the IAM group.

   - Example: `arn:aws:iam::123456789012:group/Admins`

21. **Group ID**: The unique identifier for the IAM group.

- Example: `AIDAGROUPID123`

22. **Group Creation Date**: The timestamp when the IAM group was created.

- Example: `2023-01-01T12:00:00Z`

23. **Group Members**: The IAM users that are members of the group.

- Example: `[ "john.doe", "jane.smith" ]`

MFA (Multi-Factor Authentication)

24. **MFA Devices**: Information about IAM users' MFA devices.

- Example: `{ "SerialNumber": "arn:aws:iam::123456789012:mfa/john.doe", "EnableDate": "2023-01-01T12:00:00Z" }`

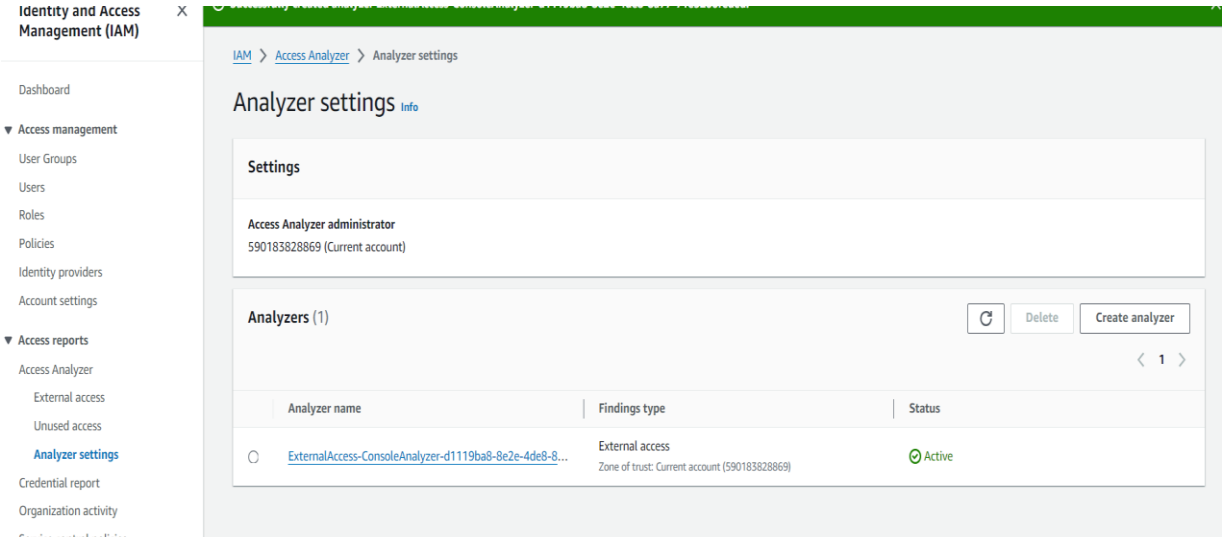
25. **MFA Status**: The status of MFA (enabled or disabled) for IAM users.

- Example: `Enabled`, `Disabled`

Access Analyzer

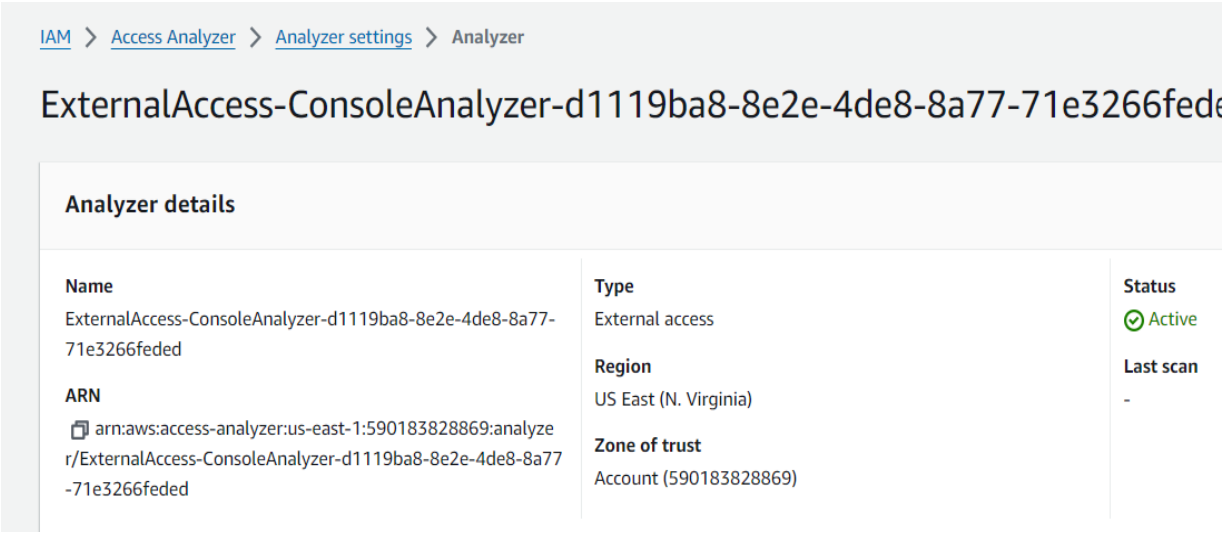
26. **Access Analyzer Name**: The name of the Access Analyzer.

- Example: `MyAccessAnalyzer`



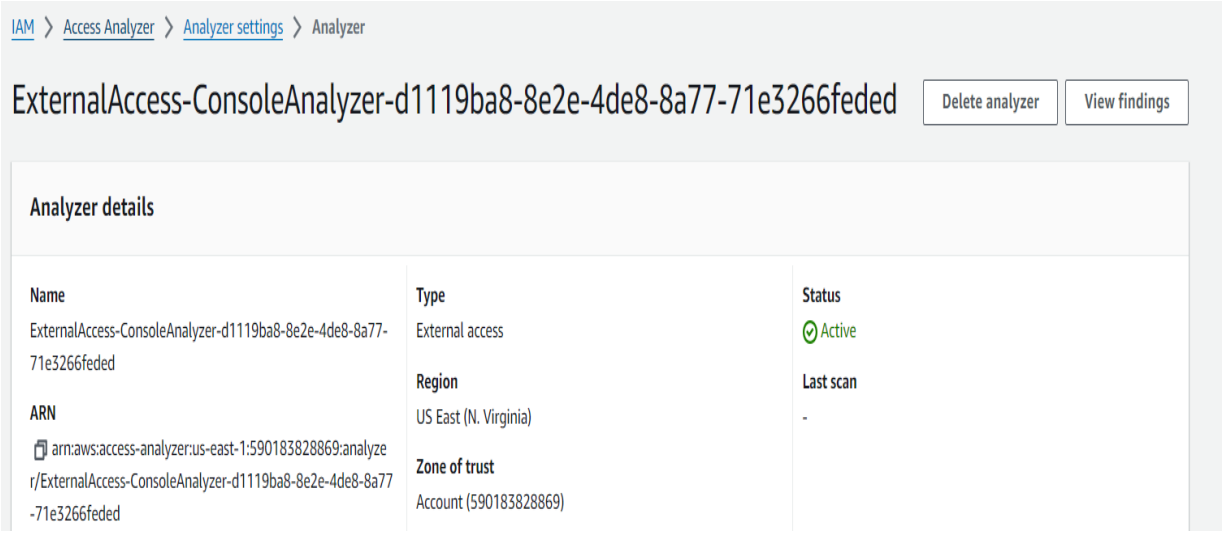
27. **Access Analyzer ARN**: The ARN of the Access Analyzer.

- Example: `arn:aws:access-analyzer:us-west-2:123456789012:analyzer/MyAccessAnalyzer`



28. **Access Analyzer Status**: The status of the Access Analyzer (active or disabled).

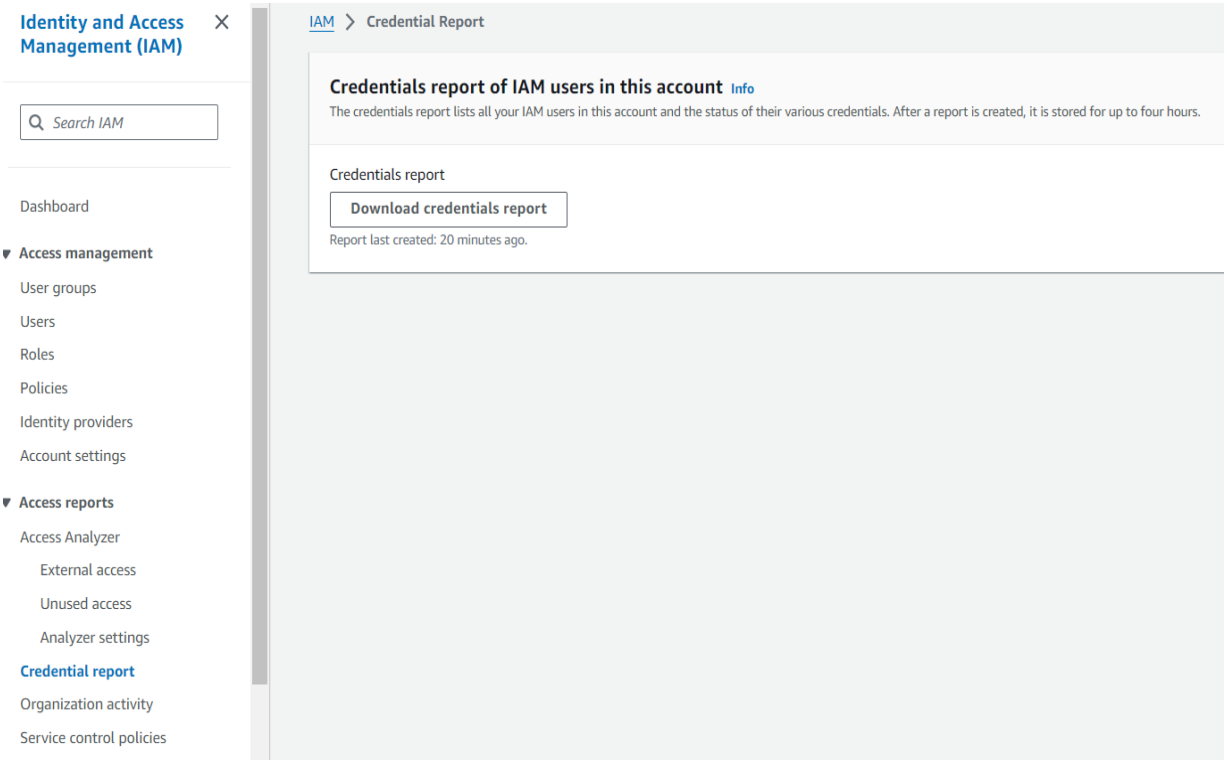
- Example: `ACTIVE`, `DISABLED`



Credential Report

29. **Credential Report**: The IAM credential report provides details about IAM users and their credentials.

- Example: `{ "user": "john.doe", "password_enabled": true, "access_key_1_active": true, "access_key_1_last_rotated": "2023-01-01T12:00:00Z" }`



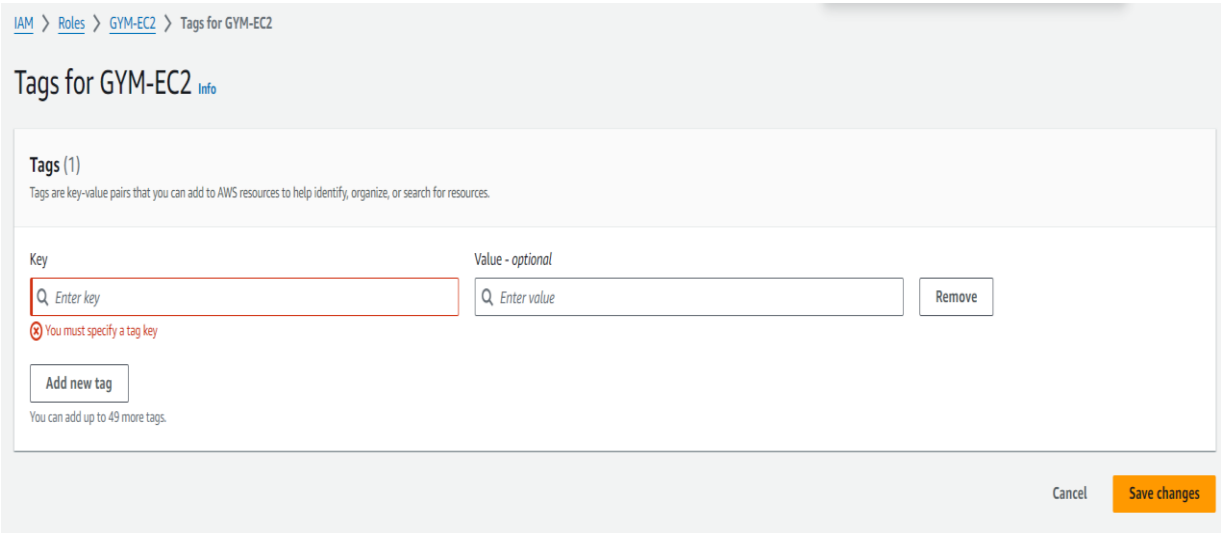
Key Rotation

30. **Password Rotation Policy**: The password rotation policy for IAM users.
   - Example: `{ "MinimumPasswordLength": 12, "RequireSymbols": true, "RequireNumbers": true }`

31. **Access Key Rotation**: The access key rotation policy for IAM users.
   - Example: `{ "AccessKeyRotation": { "RotationEnabled": true, "AutomaticallyExpireDays": 90 } }`

Tags

32. **Tags**: Key-value pairs that you can use to categorize and manage your IAM resources.
   - Example: `{ "Environment": "Production", "Department": "IT" }`



These parameters can be managed and configured through the AWS Management Console, AWS CLI, or AWS SDKs, allowing for precise control and management of access to AWS services and resources.

---

12. List of all CloudWatch parameters and their definition and description

CloudWatch is a monitoring and observability service provided by AWS that collects and tracks metrics, logs, and events from AWS resources and applications. Below is a comprehensive list of CloudWatch instance parameters along with their definitions and descriptions:

Metrics

1. **Metric Namespace**: A container for CloudWatch metrics, typically related to a specific AWS service or application.

   - Example: `AWS/EC2`, `AWS/S3`, `MyApp/Requests`

2. **Metric Name**: The name of the metric within a namespace.

   - Example: `CPUUtilization`, `RequestCount`, `ErrorCount`

3. **Dimensions**: Name-value pairs that uniquely identify a metric.

   - Example: `{ "InstanceId": "i-1234567890abcdef0" }`

4. **Timestamp**: The timestamp when the metric data point was generated.

- Example: `2023-01-01T12:00:00Z`

5. **Unit**: The unit of the metric (e.g., Count, Percent, Bytes).
   - Example: `Count`, `Percent`, `Bytes`

6. **Statistic Values**: Statistical data (e.g., Minimum, Maximum, Average) computed over a set of metric data points.
   - Example: `{ "Minimum": 0.0, "Maximum": 100.0, "Average": 50.0 }`

Alarms

7. **Alarm Name**: The name of the CloudWatch alarm.
   - Example: `HighCPUAlarm`

8. **Alarm ARN**: The Amazon Resource Name (ARN) of the CloudWatch alarm.
   - Example: `arn:aws:cloudwatch:us-west-2:123456789012:alarm/HighCPUAlarm`

9. **Alarm Description**: A description of what the alarm monitors.
   - Example: `Triggered when CPU utilization exceeds 80% for 5 minutes`

10. **Alarm State**: The current state of the alarm (e.g., OK, ALARM, INSUFFICIENT_DATA).
    - Example: `ALARM`, `OK`, `INSUFFICIENT_DATA`

11. **Metric Threshold**: The threshold value for the alarm to be triggered.
    - Example: `80.0`

12. **Actions**: The actions to be taken when the alarm state changes (e.g., send notification, trigger Auto Scaling).
    - Example: `{ "arn": "arn:aws:sns:us-west-2:123456789012:my-topic" }`

Logs

13. **Log Group Name**: The name of the CloudWatch Logs log group.

   - Example: `/aws/lambda/my-function`, `/var/log/nginx/access.log`

14. **Log Stream Name**: The name of the CloudWatch Logs log stream within a log group.

   - Example: `2023/01/01/[$LATEST]abcdef0123456789`

15. **Log Events**: Individual log entries within a log stream.

   - Example: `{ "timestamp": 1641411600000, "message": "Error: Out of memory", "ingestionTime": 1641411605000 }`

16. **Log Subscription Filters**: Filters that define which log events to route to a specific destination (e.g., Lambda function, Kinesis stream).

   - Example: `{ "filterName": "my-filter", "destinationArn": "arn:aws:lambda:us-west-2:123456789012:function:my-function" }`

Dashboards

17. **Dashboard Name**: The name of the CloudWatch dashboard.

   - Example: `MyDashboard`

18. **Dashboard Widgets**: Widgets on the CloudWatch dashboard displaying metrics, logs, or other visualizations.

   - Example: `{ "type": "metric", "x": 0, "y": 0, "width": 6, "height": 3, "properties": { "metrics": [ [ "AWS/EC2", "CPUUtilization", "InstanceId", "i-1234567890abcdef0" ] ], "view": "timeSeries" } }`

Events

19. **Event Bus Name**: The name of the CloudWatch Events event bus.

   - Example: `default`

20. **Event Rule Name**: The name of the CloudWatch Events rule.

   - Example: `MyEventRule`

21. **Event Pattern**: A pattern that identifies events to route to a target.

   - Example: `{ "source": [ "aws.ec2" ], "detail-type": [ "EC2 Instance State-change Notification" ], "detail": { "state": [ "running" ] } }`

22. **Targets**: Actions to be taken when an event matches the event pattern (e.g., invoke Lambda function, send notification).

   - Example: `{ "arn": "arn:aws:lambda:us-west-2:123456789012:function:my-function", "id": "target1" }`

Insights

23. **Insight Name**: The name of the CloudWatch Logs Insights query result.

   - Example: `HighErrorLogsInsight`

24. **Insight Query**: A query used to analyze logs and generate insights.

   - Example: `fields @timestamp, @message | filter @message like /error/`

25. **Insight Results**: Results returned by the CloudWatch Logs Insights query.

   - Example: `{ "timestamp": 1641411600000, "message": "Error: Out of memory" }`

Cross-Account Access

26. **Cross-Account Access**: Permissions that allow a resource in one AWS account to use a resource (such as a CloudWatch alarm or metric) in another account.

   - Example: `{ "accountId": "123456789012", "region": "us-west-2" }`

Permissions

27. **IAM Policies**: IAM policies that define permissions for CloudWatch resources.

   - Example: `{ "Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Action": [ "cloudwatch:GetMetricStatistics", "cloudwatch:ListMetrics" ], "Resource": "*" } ] }`

Pricing and Billing

28. **Pricing**: Pricing information for CloudWatch metrics, logs, and events.

- Example: `{ "metrics": { "pricePerUnit": "0.3 USD per metric" }, "logs": { "pricePerGB": "0.5 USD per GB" }, "events": { "pricePerEvent": "0.001 USD per event" } }`

These parameters can be managed and configured through the AWS Management Console, AWS CLI, or AWS SDKs, enabling comprehensive monitoring and observability of AWS resources and applications using CloudWatch.

---

13. List of all CloudTrail parameters and their definition and description

CloudTrail is an AWS service that enables governance, compliance, and operational and risk auditing of your AWS account. Below is a comprehensive list of CloudTrail instance parameters along with their definitions and descriptions:

Trail Configuration

1. **Trail Name**: The name of the CloudTrail trail.

- Example: `MyCloudTrail`

2. **Trail ARN**: The Amazon Resource Name (ARN) of the CloudTrail trail.

- Example: `arn:aws:cloudtrail:us-west-2:123456789012:trail/MyCloudTrail`

3. **S3 Bucket Name**: The name of the Amazon S3 bucket where CloudTrail logs are stored.

- Example: `my-cloudtrail-logs`

4. **S3 Bucket ARN**: The ARN of the Amazon S3 bucket where CloudTrail logs are stored.

- Example: `arn:aws:s3:::my-cloudtrail-logs`

5. **Log File Prefix**: The prefix added to log file names stored in the S3 bucket.

- Example: `cloudtrail/logs`

6. **Include Global Service Events**: Indicates whether CloudTrail includes global service events (such as IAM actions) in the trail.

- Example: `true`, `false`

7. **Is Multi-Region Trail**: Indicates whether the trail is a multi-region trail.

- Example: `true`, `false`

8. **Home Region**: The AWS region where the CloudTrail trail is created.

- Example: `us-west-2`

9. **Trail Status**: The current status of the CloudTrail trail (e.g., `Creating`, `Active`, `Updating`, `Deleted`).

- Example: `Active`

10. **CloudWatch Logs Log Group ARN**: The ARN of the CloudWatch Logs log group where CloudTrail logs are delivered.

- Example: `arn:aws:logs:us-west-2:123456789012:log-group:CloudTrail/DefaultLogGroup`

11. **Event Selectors**: Configures which events are logged for the trail (e.g., data events, management events).

- Example: `{ "DataResources": [ { "Type": "AWS::S3::Object", "Values": [ "arn:aws:s3:::my-bucket/*" ] } ], "IncludeManagementEvents": true, "ReadWriteType": "All" }`

12. **CloudWatch Logs Role ARN**: The ARN of the IAM role that grants CloudTrail permission to write logs to CloudWatch Logs.

- Example: `arn:aws:iam::123456789012:role/CloudTrail_CloudWatchLogs_Role`

13. **KMS Key ID**: The AWS Key Management Service (KMS) key ID used to encrypt CloudTrail logs.

- Example: `arn:aws:kms:us-west-2:123456789012:key/abcd-1234-efgh-5678`

Management and Access

14. **Trail Creation Time**: The timestamp when the CloudTrail trail was created.

- Example: `2023-01-01T12:00:00Z`

15. **Last Updated Time**: The timestamp when the CloudTrail trail was last updated.

   - Example: `2023-01-02T12:00:00Z`

16. **Owner**: The AWS account ID that owns the CloudTrail trail.

   - Example: `123456789012`

17. **Trail Insights Configuration**: Configuration that enables or disables CloudTrail Insights.

   - Example: `{ "InsightSelectors": [ { "InsightType": "ApiCallRateInsight", "InsightEnabled": true } ] }`

Security and Compliance

18. **CloudTrail Policy**: The IAM policy that defines permissions for CloudTrail trails.

   - Example: `{ "Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Action": [ "cloudtrail:CreateTrail", "cloudtrail:UpdateTrail", "cloudtrail:DeleteTrail" ], "Resource": "*" } ] }`

19. **CloudTrail Tags**: Tags assigned to the CloudTrail trail.

   - Example: `{ "Environment": "Production", "Owner": "DevOpsTeam" }`

20. **Event History**: Details about events captured by the CloudTrail trail, including management and data events.

   - Example: `{ "EventId": "01234567-89ab-cdef-0123-456789abcdef", "EventName": "CreateBucket", "EventTime": "2023-01-01T12:00:00Z", "Resources": [ { "ResourceType": "AWS::S3::Bucket", "ResourceName": "my-bucket" } ] }`

Integration and Monitoring

21. **CloudWatch Integration**: Configuration for delivering CloudTrail logs to CloudWatch Logs.

   - Example: `{ "LogGroupArn": "arn:aws:logs:us-west-2:123456789012:log-group:CloudTrail/DefaultLogGroup", "RoleArn": "arn:aws:iam::123456789012:role/CloudTrail_CloudWatchLogs_Role" }`

22. **CloudTrail Insights Integration**: Integration with CloudTrail Insights for analyzing and visualizing CloudTrail logs.

- Example: `{ "InsightSelectors": [ { "InsightType": "ApiCallRateInsight", "InsightEnabled": true } ] }`

23. **Amazon SNS Integration**: Integration with Amazon SNS for receiving notifications about CloudTrail events.
    - Example: `{ "TopicArn": "arn:aws:sns:us-west-2:123456789012:my-cloudtrail-notifications" }`

24. **Event History Integration**: Integration with Amazon EventBridge for processing CloudTrail events.
    - Example: `{ "RuleArn": "arn:aws:events:us-west-2:123456789012:rule/ProcessCloudTrailEvents" }`

Pricing and Billing

25. **Pricing**: Pricing information for CloudTrail, including free tier allowances and additional costs.
    - Example: `{ "freeTierEventsPerAccount": 100000, "paidEventsPerAccount": "0.10 USD per 100,000 events" }`

Compliance and Auditing

26. **Compliance Reports**: Reports generated by CloudTrail for compliance audits and governance.
    - Example: `{ "S3BucketName": "my-compliance-reports", "S3KeyPrefix": "cloudtrail-reports" }`

These parameters can be managed and configured through the AWS Management Console, AWS CLI, or AWS SDKs, providing comprehensive auditing and governance capabilities for your AWS account and resources using CloudTrail.

---

14. List of all CodeDeploy parameters and their definition and description

CodeDeploy is an AWS service that automates software deployments to various compute services like EC2 instances, Lambda functions, or even on-premises servers. Below is a comprehensive list of CodeDeploy instance parameters along with their definitions and descriptions:

Applications and Deployments

1. **Application Name**: The name of the CodeDeploy application.

   - Example: `MyApp`

2. **Deployment Group Name**: The name of the deployment group within a CodeDeploy application.

   - Example: `ProductionGroup`

3. **Deployment ID**: The unique identifier for a deployment instance.

   - Example: `d-1A2B3C4D5E`

4. **Deployment Status**: The current status of the deployment (e.g., `Created`, `InProgress`, `Succeeded`, `Failed`).

   - Example: `Succeeded`

5. **Deployment Configuration**: The configuration settings for a deployment (e.g., deployment strategy, traffic shifting).

   - Example: `CodeDeployDefault.OneAtATime`

6. **Deployment Type**: The type of deployment (e.g., In-place, Blue/green).

   - Example: `BLUE_GREEN`

7. **Application Revision**: Details about the application revision being deployed (e.g., source location, revision type).

   - Example: `{ "revisionType": "S3", "s3Location": { "bucket": "my-bucket", "key": "my-app.zip", "bundleType": "zip" } }`

Deployment Configuration

8. **Traffic Routing Configuration**: Configuration settings for traffic shifting during deployments (for blue/green deployments).

   - Example: `{ "type": "TimeBasedLinear", "timeBasedLinear": { "linearIntervalPercentage": 10, "interval": 1 } }`

9. **Auto Scaling Groups**: Information about Auto Scaling groups used in deployments.

  - Example: `[ "my-asg-1", "my-asg-2" ]`

10. **Deployment Timeout**: The maximum time (in minutes) CodeDeploy waits for a deployment to complete.

  - Example: `60`

11. **Deployment Settings**: Additional settings for a deployment (e.g., rollback configuration).

  - Example: `{ "rollback": { "deploymentType": "LAST_SUCCESSFUL" } }`

Hooks and Events

12. **Lifecycle Events**: Events triggered during a deployment lifecycle (e.g., `ApplicationStop`, `BeforeInstall`, `AfterInstall`).

  - Example: `[ "ApplicationStop", "BeforeInstall", "AfterInstall" ]`

13. **Deployment Hooks**: Scripts or commands to run at different stages of a deployment.

  - Example: `{ "BeforeBlockTraffic": { "path": "scripts/before_block_traffic.sh", "timeout": 300, "runOrder": 1 } }`

Resources and Permissions

14. **IAM Role**: The IAM role used by CodeDeploy to perform deployments.

  - Example: `arn:aws:iam::123456789012:role/CodeDeployServiceRole`

15. **Instance Tags**: Tags assigned to EC2 instances or other compute resources targeted by deployments.

  - Example: `{ "Environment": "Production", "Application": "MyApp" }`

Monitoring and Logs

16. **Deployment Logs**: Logs and details about deployment actions and events.

- Example: `{ "startTime": "2023-01-01T12:00:00Z", "endTime": "2023-01-01T12:30:00Z", "status": "Succeeded" }`

17. **Deployment Metrics**: Metrics and performance data related to deployments.
   - Example: `{ "ErrorCount": 0, "RetryCount": 0 }`

Integration and Notifications

18. **Amazon SNS Integration**: Integration with Amazon SNS for receiving deployment notifications.
   - Example: `{ "TopicARN": "arn:aws:sns:us-west-2:123456789012:CodeDeployNotifications" }`

19. **GitHub Integration**: Integration with GitHub for source code repositories.
   - Example: `{ "repository": "my-org/my-repo", "commitId": "abcdef1234567890" }`

Compliance and Auditing

20. **Deployment History**: Historical data and logs of previous deployments.
   - Example: `{ "deploymentId": "d-1A2B3C4D5E", "revision": "my-app.zip", "status": "Succeeded" }`

Pricing and Billing

21. **Pricing**: Pricing information for CodeDeploy, including any applicable free tier and pricing details.
   - Example: `{ "freeTierEventsPerMonth": 5000, "pricePerDeployedInstance": "0.02 USD per instance-hour" }`

These parameters can be managed and configured through the AWS Management Console, AWS CLI, or AWS SDKs, facilitating automated and controlled deployments of applications across various AWS compute services using CodeDeploy.

---

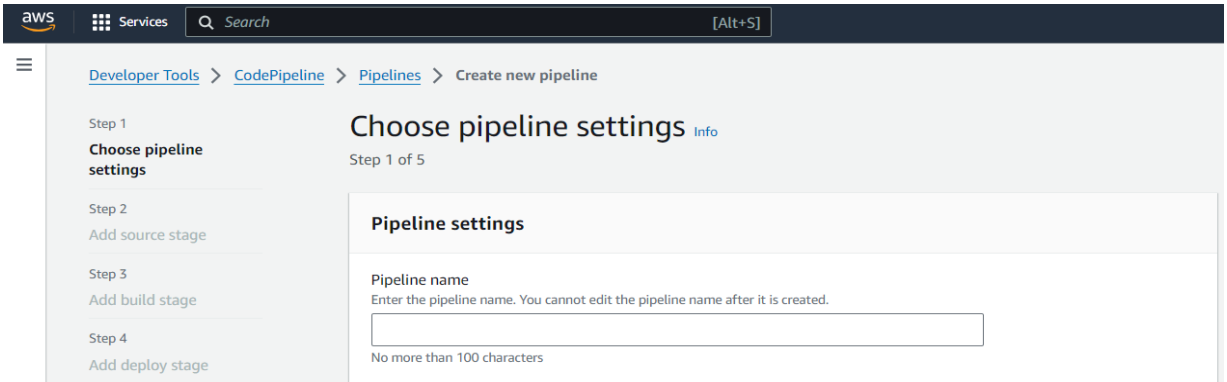
15. List of all CodePipeline parameters and their definition and description

CodePipeline is a fully managed continuous integration and continuous delivery (CI/CD) service provided by AWS. It automates the build, test, and deploy phases of your release process every time there is a code change. Below is a comprehensive list of CodePipeline instance parameters along with their definitions and descriptions:

Pipeline Configuration

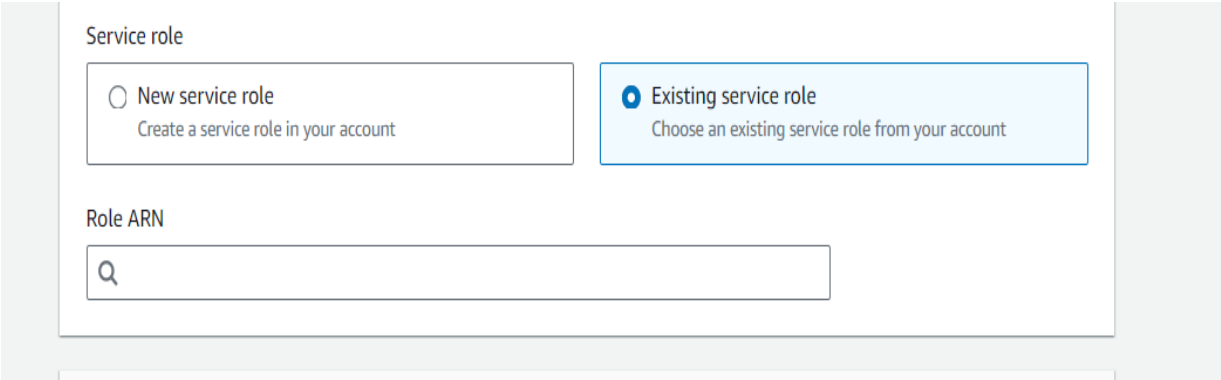
1. **Pipeline Name**: The name of the CodePipeline pipeline.

   - Example: `MyPipeline`



2. **Pipeline ARN**: The Amazon Resource Name (ARN) of the CodePipeline pipeline.

   - Example: `arn:aws:codepipeline:us-west-2:123456789012:MyPipeline`



3. **Pipeline Stages**: The sequential stages in the pipeline (e.g., Source, Build, Deploy).

   - Example: `[ "Source", "Build", "Deploy" ]`

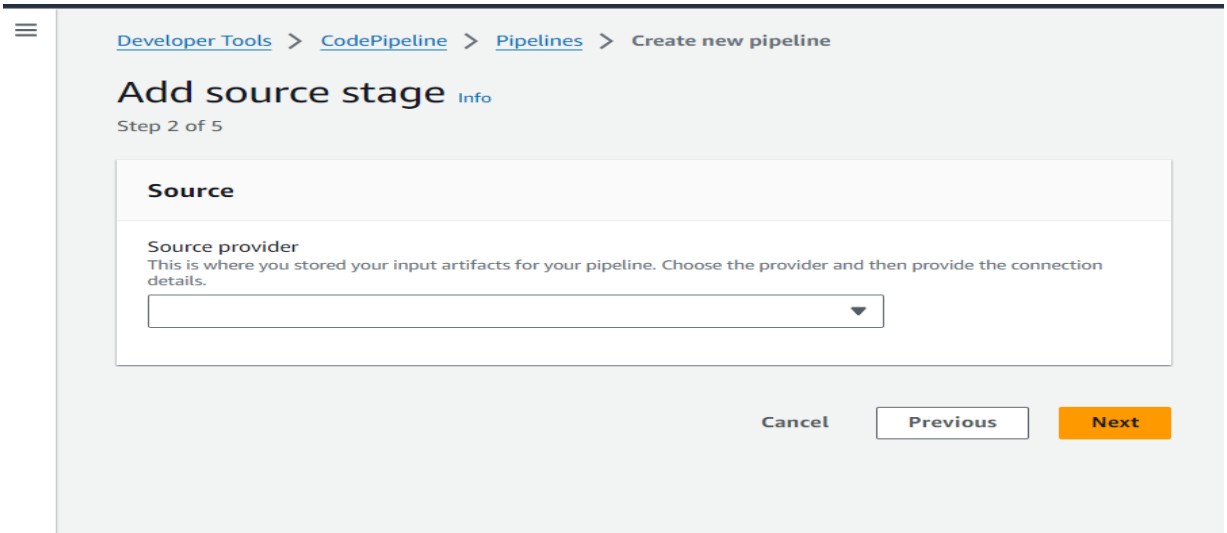
4. **Pipeline Actions**: Actions configured within each stage to perform specific tasks (e.g., source code checkout, build artifact creation, deployment).

   - Example: `{ "name": "Source", "category": "Source", "provider": "GitHub", "outputArtifacts": [ { "name": "SourceArtifact" } ], "configuration": { "Owner": "my-org", "Repo": "my-repo", "Branch": "main" } }`

Source Control

5. **Source Provider**: The source code repository provider (e.g., GitHub, AWS CodeCommit).

   - Example: `GitHub`



6. **Source Location**: The location of the source code repository (e.g., repository URL, branch).

   - Example: `{ "Owner": "my-org", "Repo": "my-repo", "Branch": "main" }`

7. **Source Actions**: Actions related to retrieving source code and initiating the pipeline.

   - Example: `{ "category": "Source", "name": "Source", "provider": "GitHub", "outputArtifacts": [ { "name": "SourceArtifact" } ], "configuration": { "Owner": "my-org", "Repo": "my-repo", "Branch": "main" } }`

Build and Test

8. **Build Provider**: The build provider or service used to compile code, run tests, and generate artifacts.

   - Example: `AWS CodeBuild`

9. **Build Environment**: Configuration for the environment where build actions are executed (e.g., compute type, environment variables).

   - Example: `{ "computeType": "BUILD_GENERAL1_SMALL", "image": "aws/codebuild/standard:4.0", "environmentVariables": [ { "name": "ENVIRONMENT", "value": "Production" } ] }`

10. **Build Actions**: Actions related to building and testing code.

   - Example: `{ "category": "Build", "name": "Build", "provider": "AWS CodeBuild", "inputArtifacts": [ { "name": "SourceArtifact" } ], "outputArtifacts": [ { "name": "BuildArtifact" } ], "configuration": { "ProjectName": "MyCodeBuildProject" } }`

Deploy and Release

11. **Deployment Provider**: The deployment provider or service used to deploy artifacts (e.g., AWS CodeDeploy, AWS Elastic Beanstalk).

   - Example: `AWS CodeDeploy`

12. **Deployment Actions**: Actions related to deploying artifacts to production or staging environments.

   - Example: `{ "category": "Deploy", "name": "Deploy", "provider": "AWS CodeDeploy", "inputArtifacts": [ { "name": "BuildArtifact" } ], "configuration": { "ApplicationName": "MyApp", "DeploymentGroupName": "ProductionGroup" } }`

Notifications and Integrations

13. **Notification Actions**: Actions to notify stakeholders or systems about pipeline events (e.g., success, failure).

   - Example: `{ "category": "Approval", "name": "Notify", "provider": "SNS", "configuration": { "TopicArn": "arn:aws:sns:us-west-2:123456789012:PipelineNotifications" } }`

14. **Integration Actions**: Actions to integrate with external services or tools during pipeline execution.

   - Example: `{ "category": "Invoke", "name": "Integration", "provider": "Lambda", "configuration": { "FunctionName": "MyLambdaFunction" } }`

Monitoring and Logs

15. **Pipeline Execution Logs**: Logs and details about each pipeline execution (e.g., start time, end time, status).

   - Example: `{ "startTime": "2023-01-01T12:00:00Z", "endTime": "2023-01-01T12:30:00Z", "status": "Succeeded" }`

16. **Execution Artifacts**: Artifacts generated during pipeline execution (e.g., build artifacts, deployment packages).

   - Example: `{ "name": "BuildArtifact", "location": "s3://my-bucket/build/my-app.zip" }`

Permissions and Security

17. **IAM Role**: The IAM role used by CodePipeline to execute pipeline actions.

   - Example: `arn:aws:iam::123456789012:role/CodePipelineServiceRole`

18. **Resource Permissions**: Permissions and policies that define access to AWS resources used by pipeline actions.

   - Example: `{ "Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Action": [ "s3:GetObject", "s3:PutObject" ], "Resource": "arn:aws:s3:::my-bucket/*" } ] }`

Compliance and Auditing

19. **Pipeline History**: Historical data and logs of previous pipeline executions.

   - Example: `{ "pipelineExecutionId": "12345678-abcd-5678-efgh-1234567890ab", "status": "Succeeded", "artifactRevisions": [ { "name": "SourceArtifact", "revisionId": "abcdef1234567890" } ] }`

Pricing and Billing

20. **Pricing**: Pricing information for CodePipeline, including any applicable free tier and pricing details.

   - Example: `{ "freeTierPipelinesPerMonth": 1, "pricePerPipelineExecution": "0.75 USD per pipeline execution" }`

These parameters can be managed and configured through the AWS Management Console, AWS CLI, or AWS SDKs, enabling automated software release processes using CodePipeline across various AWS services and external integrations.

****